Waffy Database

Unified schema, updated decisions, relationships, and computed query catalog

Purpose: This file documents the final logical database design for Waffy after the latest project changes, including clinic-managed doctor onboarding, complaint routing to multiple admin receivers, and the mandatory 5 JOD appointment deposit workflow.

Final design decision	Applied rule
Currency	JOD only; no Currency column.
Payments	Mock-only payments; no external gateway integration.
Computed data	Totals, averages, counts, age, and dashboard metrics are query-based by default.
CSV fields	Traits, WorkingDays, AcceptedPayments, and some PetType/working-day lists remain CSV for academic scope.
Owner data	Store and clinic owner identity stays in Users; no duplicated owner profile table.
Subscriptions	Subscription logic is centralized in Subscription and SubscriptionPlan only.
Doctor onboarding	Clinic creates doctor accounts directly; InviteCode is removed.
Complaints routing	Clinic/store complaints are routed to entity admin plus SuperAdmin; system complaints go to SuperAdmin only.
Appointment guarantee	Every appointment holds a 5 JOD deposit; refund on attendance, forfeit on no-show.
Polymorphic references	Allowed in Subscription, WalletTransaction, Review targets, AdoptionPost, and other approved cases.

Entities, attributes, types, and design purpose

Users

Why this entity exists: Core account table. Every actor in Waffy starts here: pet owner, clinic owner, doctor, store owner, or super admin.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Primary identifier for each user.
Email	NVARCHAR(150), NOT NULL, UNIQUE	Login email and unique contact identity.
PasswordHash	NVARCHAR(255), NOT NULL	Stores the hashed password securely.
Phone	NVARCHAR(20), NOT NULL	Primary phone number used in the platform.

IsPhoneVerified	BIT, DEFAULT 0	Marks whether the phone number has been verified.
IsEmailVerified	BIT, DEFAULT 0	Marks whether the email address has been verified.
FullName	NVARCHAR(100), NOT NULL	Display name and legal/basic account name.
City	NVARCHAR(100), NOT NULL	User city for service matching and search.
ProfileImage	NVARCHAR(500), NULL	Profile image path or URL.
Area	NVARCHAR(100), NULL	More specific location within the city.
AddressDetail	NVARCHAR(300), NULL	Optional detailed address.
Latitude	DECIMAL(9,6), NULL	Map latitude for location-based features.
Longitude	DECIMAL(9,6), NULL	Map longitude for location-based features.
WhatsAppPhone	NVARCHAR(20), NULL	Optional WhatsApp contact number.
BirthDate	DATE, NULL	Used for optional profile information.
Gender	NVARCHAR(20), NULL	Optional profile field.
Bio	NVARCHAR(500), NULL	Short profile description.
AccountStatus	NVARCHAR(20), NOT NULL, DEFAULT 'Active'	Controls whether the account is active, suspended, or banned.
WarningCount	INT, DEFAULT 0	Tracks moderation warnings.
SuspensionReason	NVARCHAR(500), NULL	Reason shown when the account is suspended.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Account creation timestamp.
LastActiveAt	DATETIME2, NULL	Latest known platform activity time.

Relationships: Users 1—Many UserRole; Users 1—1 Wallet; Users 1—1 Cart; Users 1—Many Pet, Appointment, Complaint, Donation, Notification, Review; Users 1—0/1 Clinic, Store, Doctor depending on role

Important note: Recommended CHECK: WarningCount >= 0.

Important note: Use this table as the single source of owner/admin identity across the system.

Role

Why this entity exists: Lookup table for role names used in authorization and dashboard access control.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Role identifier.
Name	NVARCHAR(50), NOT NULL, UNIQUE	Role label such as PetOwner, ClinicOwner, Doctor, StoreOwner, or

		SuperAdmin.

Relationships: Role 1—Many UserRole

UserRole

Why this entity exists: Bridge table between Users and Role so one user can hold multiple roles when needed.

Attribute	Type	Note / Why needed
UserId	INT, FK → Users.Id	User who owns the role.
RoleId	INT, FK → Role.Id	Assigned role.
AssignedAt	DATETIME2, DEFAULT GETDATE()	When the role was assigned.

Relationships: Users Many—Many Role through UserRole

Important note: Primary Key = (UserId, RoleId).

Wallet

Why this entity exists: One-to-one financial container for each user. Used for rewards, refunds, and internal payments.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Wallet identifier.
UserId	INT, FK → Users.Id, UNIQUE	Each user has at most one wallet.
Balance	DECIMAL(10,2), DEFAULT 0.00	Current cached balance for fast display.
Status	NVARCHAR(20), DEFAULT 'Active'	Wallet state, such as Active or Suspended.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Wallet creation timestamp.
UpdatedAt	DATETIME2, NULL	Latest balance/status update time.

Relationships: Users 1—1 Wallet; Wallet 1—Many WalletTransaction

Important note: Balance can be recalculated from WalletTransaction when needed.

WalletTransaction

Why this entity exists: Detailed transaction log for wallet activity. Also supports appointment deposit payment/refund tracking through polymorphic references.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Transaction identifier.
WalletId	INT, FK → Wallet.Id	Wallet that owns this transaction.
TransactionCode	NVARCHAR(50), NULL, UNIQUE	Human/system-friendly tracking code.
Type	NVARCHAR(30), NOT NULL	Deposit, Withdraw, Payment, Cashback, Reward, Refund, or Fee.

Amount	DECIMAL(10,2), NOT NULL	Transaction amount.
PaymentMethod	NVARCHAR(20), NULL	Card, CliQ, Cash, Wallet, or another mock method.
Description	NVARCHAR(300), NULL	Short explanation shown in history.
ReferenceType	NVARCHAR(50), NULL	Optional related entity type, such as Order, Appointment, LostPet, or Donation.
ReferenceId	INT, NULL	Identifier of the referenced entity.
Status	NVARCHAR(20), DEFAULT 'Completed'	Completed, Pending, or Failed.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Transaction timestamp.

Relationships: Wallet 1—Many WalletTransaction

Important note: Recommended CHECK: Amount > 0.

Pet

Why this entity exists: Pet profile table. Used across appointments, adoption, lost-pet reports, and owner profiles.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Pet identifier.
OwnerId	INT, FK → Users.Id, NOT NULL	User who owns the pet.
Name	NVARCHAR(100), NOT NULL	Pet name.
Species	NVARCHAR(50), NOT NULL	Dog, Cat, Bird, Rabbit, or Other.
Breed	NVARCHAR(100), NULL	Breed or subtype.
Gender	NVARCHAR(10), NOT NULL	Pet gender.
BirthDate	DATE, NULL	Used to calculate pet age.
Color	NVARCHAR(100), NULL	Visual color description.
MicrochipId	NVARCHAR(50), NULL	Optional microchip reference; should be uniquely indexed when not null.
Weight	DECIMAL(5,2), NULL	Pet weight.
VaccinationStatus	NVARCHAR(20), NULL	Vaccinated, NotVaccinated, or Partial.
IsNeutered	BIT, DEFAULT 0	Whether the pet is neutered/spayed.
MedicalNotes	NVARCHAR(1000), NULL	Important health notes.
Traits	NVARCHAR(500), NULL	Behavior/personality traits stored as CSV by design.
EnergyLevel	NVARCHAR(20), NULL	Low, Medium, or High.

GoodWithChildren	BIT, NULL	Compatibility with children.
GoodWithPets	BIT, NULL	Compatibility with other pets.
IsTrained	BIT, DEFAULT 0	Whether the pet has training.
ImageUrl	NVARCHAR(500), NULL	Pet image.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Record creation timestamp.

Relationships: Users 1—Many Pet; Pet 1—Many Appointment; Pet 1—Many LostPetReport and AdoptionPost (optional)

Clinic

Why this entity exists: Main clinic business entity. Holds operational, contact, and map information for a veterinary clinic.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Clinic identifier.
OwnerUserId	INT, FK → Users.Id, UNIQUE	Clinic owner account; one user owns at most one clinic.
Name	NVARCHAR(150), NOT NULL	Clinic name.
ClinicType	NVARCHAR(50), NULL	Private, Hospital, Mobile, or Center.
Phone	NVARCHAR(20), NOT NULL	Main clinic phone.
Email	NVARCHAR(150), NULL	Clinic email.
Description	NVARCHAR(1000), NULL	Clinic overview.
City	NVARCHAR(100), NOT NULL	City used for search and filtering.
Area	NVARCHAR(100), NULL	Area within the city.
Address	NVARCHAR(300), NULL	Street or detailed address.
Latitude	DECIMAL(9,6), NULL	Latitude for map placement.
Longitude	DECIMAL(9,6), NULL	Longitude for map placement.
DefaultOpenTime	TIME, NULL	Default opening time.
DefaultCloseTime	TIME, NULL	Default closing time.
HasEmergency	BIT, DEFAULT 0	Whether emergency service is available.
VerificationStatus	NVARCHAR(20), DEFAULT 'Pending'	Pending, Approved, Rejected, or Suspended.
Rating	DECIMAL(3,2), DEFAULT 0.00	Cached rating value for quick display.
LogoUrl	NVARCHAR(500), NULL	Clinic logo.
CoverImageUrl	NVARCHAR(500), NULL	Clinic cover image.

CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—1 Clinic; Clinic 1—Many Doctor, ClinicSchedule, ClinicService, ClinicDocument, Appointment, Complaint

Important note: SubscriptionPlan and SubscriptionExpiry were deliberately removed; subscription is now handled only in Subscription.

ClinicSchedule

Why this entity exists: Per-day clinic opening schedule. Allows each day to have a different working window.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Schedule row identifier.
ClinicId	INT, FK → Clinic.Id	Clinic that owns the schedule row.
DayOfWeek	INT, NOT NULL	0 to 6, representing Sunday to Saturday.
OpenTime	TIME, NOT NULL	Opening time for that day.
CloseTime	TIME, NOT NULL	Closing time for that day.
IsWorkingDay	BIT, DEFAULT 1	Marks whether that day is actually active.

Relationships: Clinic 1—Many ClinicSchedule

Important note: Recommended UNIQUE: (ClinicId, DayOfWeek).

Important note: Recommended CHECK: DayOfWeek BETWEEN 0 AND 6 and OpenTime < CloseTime.

ClinicService

Why this entity exists: Clinic-level service catalog used to list what the clinic can offer.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Clinic service identifier.
ClinicId	INT, FK → Clinic.Id	Owner clinic.
ServiceName	NVARCHAR(100), NOT NULL	Service label.
IsActive	BIT, DEFAULT 1	Whether the service is currently offered.

Relationships: Clinic 1—Many ClinicService

ClinicDocument

Why this entity exists: Verification and legal document storage for clinic approval and compliance.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Document identifier.
ClinicId	INT, FK → Clinic.Id	Clinic that uploaded the document.
DocumentType	NVARCHAR(50), NOT NULL	MunicipalLicense, PracticeLicense,

		OwnerID, Logo, etc.
FileUrl	NVARCHAR(500), NOT NULL	Stored file path or URL.
Status	NVARCHAR(20), DEFAULT 'Pending'	Pending, Approved, Rejected, or Expired.
ExpiryDate	DATE, NULL	Document expiry date if applicable.
UploadedAt	DATETIME2, DEFAULT GETDATE()	Upload timestamp.

Relationships: Clinic 1—Many ClinicDocument

Doctor

Why this entity exists: Doctor profile table. The clinic creates the doctor account and attaches it to a clinic; doctors no longer join using an invite code.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Doctor identifier.
UserId	INT, FK → Users.Id, UNIQUE	The user account that represents the doctor.
ClinicId	INT, FK → Clinic.Id	Clinic the doctor works under.
Specialization	NVARCHAR(100), NOT NULL	Primary specialization.
SecondarySpec	NVARCHAR(100), NULL	Optional secondary specialization.
ExperienceYears	INT, NULL	Years of experience.
Qualification	NVARCHAR(300), NULL	Degree or qualification.
Certifications	NVARCHAR(1000), NULL	Certificates or licenses summary.
Bio	NVARCHAR(500), NULL	Doctor biography.
Languages	NVARCHAR(200), NULL	Languages spoken.
WorkingDays	NVARCHAR(50), NULL	Working-day list stored as CSV for project simplicity.
StartTime	TIME, NULL	Daily shift start time.
EndTime	TIME, NULL	Daily shift end time.
AppointmentDuration	INT, DEFAULT 30	Appointment duration in minutes.
MaxDailyAppointments	INT, DEFAULT 12	Max number of appointments per day.
IsAvailable	BIT, DEFAULT 1	Availability toggle.
VerificationStatus	NVARCHAR(20), DEFAULT 'Pending'	Pending, Approved, or Rejected.
Rating	DECIMAL(3,2), DEFAULT 0.00	Cached average rating.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—1 Doctor; Clinic 1—Many Doctor; Doctor 1—Many DoctorService, DoctorDocument, Appointment

Important note: InviteCode was removed deliberately after the sign-in flow changed.

Important note: Recommended CHECK: ExperienceYears >= 0 and AppointmentDuration > 0.

DoctorService

Why this entity exists: Doctor-specific services and pricing. Supports services that differ from the general clinic service list.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Doctor service identifier.
DoctorId	INT, FK → Doctor.Id	Doctor who provides the service.
ServiceName	NVARCHAR(100), NOT NULL	Service label.
Price	DECIMAL(10,2), NULL	Optional doctor-specific price.
IsActive	BIT, DEFAULT 1	Whether this service is currently enabled.

Relationships: Doctor 1—Many DoctorService

DoctorDocument

Why this entity exists: Verification and credential documents stored for each doctor.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Document identifier.
DoctorId	INT, FK → Doctor.Id	Related doctor.
DocumentType	NVARCHAR(50), NOT NULL	PracticeLicense, NationalID, Degree, etc.
FileUrl	NVARCHAR(500), NOT NULL	File path or URL.
Status	NVARCHAR(20), DEFAULT 'Pending'	Pending, Approved, Rejected, or Expired.
ExpiryDate	DATE, NULL	Document expiry date when relevant.
UploadedAt	DATETIME2, DEFAULT GETDATE()	Upload timestamp.

Relationships: Doctor 1—Many DoctorDocument

Appointment

Why this entity exists: Central booking table for clinic visits. Stores the 5 JOD attendance guarantee and its final outcome.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Appointment identifier.
UserId	INT, FK → Users.Id	User who booked the appointment.

PetId	INT, FK → Pet.Id	Pet linked to the booking.
DoctorId	INT, FK → Doctor.Id	Assigned doctor.
ClinicId	INT, FK → Clinic.Id	Assigned clinic.
AppointmentDate	DATE, NOT NULL	Scheduled date.
StartTime	TIME, NOT NULL	Appointment start time.
EndTime	TIME, NOT NULL	Appointment end time.
ServiceName	NVARCHAR(100), NULL	Service name snapshot stored at booking time.
Status	NVARCHAR(20), DEFAULT 'Pending'	Pending, Confirmed, Rejected, Completed, Cancelled, or NoShow.
AppointmentType	NVARCHAR(30), NULL	Checkup, Vaccination, Surgery, etc.
UrgencyLevel	NVARCHAR(20), DEFAULT 'Normal'	Normal, Urgent, or Emergency.
Reason	NVARCHAR(500), NULL	Reason for the visit.
Notes	NVARCHAR(500), NULL	Extra notes.
ConsultationFee	DECIMAL(10,2), NULL	Consultation or treatment fee.
PaymentMethod	NVARCHAR(20), NULL	Wallet, Card, or Cash.
PaymentStatus	NVARCHAR(20), DEFAULT 'Pending'	Pending, Paid, or Refunded.
BookingDeposit	DECIMAL(10,2), NOT NULL, DEFAULT 5.00	Mandatory attendance guarantee paid at booking.
DepositStatus	NVARCHAR(20), DEFAULT 'Held'	Held, Refunded, or Forfeited.
CancellationReason	NVARCHAR(300), NULL	Reason for cancellation if applicable.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—Many Appointment; Pet 1—Many Appointment; Doctor 1—Many Appointment; Clinic 1—Many Appointment; Appointment is referenced by WalletTransaction through ReferenceType/ReferenceId

Important note: Business rule: refund deposit if attendance is confirmed; forfeit deposit on no-show.

Important note: Recommended CHECK: StartTime < EndTime, BookingDeposit >= 0, DepositStatus IN ('Held','Refunded','Forfeited').

Store

Why this entity exists: Main store business entity. Holds shop, contact, delivery, and map information.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Store identifier.
OwnerUserId	INT, FK → Users.Id, UNIQUE	Store owner account; one user owns at most one store.

Name	NVARCHAR(150), NOT NULL	Store name.
StoreType	NVARCHAR(100), NULL	Store type or specialty.
Description	NVARCHAR(1000), NULL	Store overview.
LogoUrl	NVARCHAR(500), NULL	Store logo.
CoverImageUrl	NVARCHAR(500), NULL	Store cover image.
Email	NVARCHAR(150), NULL	Store email.
Phone	NVARCHAR(20), NULL	Store phone.
WhatsApp	NVARCHAR(20), NULL	Store WhatsApp number.
Country	NVARCHAR(100), DEFAULT 'الأردن'	Country; fixed to Jordan by default.
City	NVARCHAR(100), NOT NULL	Store city.
Area	NVARCHAR(100), NULL	Store area.
Address	NVARCHAR(300), NULL	Detailed address.
Latitude	DECIMAL(9,6), NULL	Map latitude.
Longitude	DECIMAL(9,6), NULL	Map longitude.
DeliveryAvailable	BIT, DEFAULT 0	Whether delivery is offered.
DeliveryAreas	NVARCHAR(500), NULL	Delivery zones list.
DeliveryFee	DECIMAL(10,2), NULL	Delivery cost.
DeliveryTime	NVARCHAR(100), NULL	Expected delivery time.
OpenTime	TIME, NULL	Opening time.
CloseTime	TIME, NULL	Closing time.
WorkingDays	NVARCHAR(50), NULL	Working days stored as CSV.
AcceptedPayments	NVARCHAR(200), NULL	Accepted payment methods stored as CSV.
VerificationStatus	NVARCHAR(20), DEFAULT 'Pending'	Pending, Approved, Rejected, or Suspended.
VerifiedAt	DATETIME2, NULL	When the store was verified.
VerifiedByAdminId	INT, FK → Users.Id, NULL	Admin who verified the store.
Rating	DECIMAL(3,2), DEFAULT 0.00	Cached rating value.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—1 Store; Store 1—Many StoreDocument, Category, Product, Order, Complaint

Important note: SubscriptionModel, SubscriptionPlan, SubscriptionExpiry, and CommissionRate were removed intentionally; use Subscription and SubscriptionPlan instead.

StoreDocument

Why this entity exists: Verification and legal paperwork for store approval.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Document identifier.
StoreId	INT, FK → Store.Id	Store that owns the document.
DocumentType	NVARCHAR(50), NOT NULL	CommercialRegistration, BusinessLicense, Trademark, VATCertificate, etc.
FileUrl	NVARCHAR(500), NOT NULL	File path or URL.
Status	NVARCHAR(20), DEFAULT 'Pending'	Pending, Approved, Rejected, or Expired.
ExpiryDate	DATE, NULL	Expiry date where relevant.
UploadedAt	DATETIME2, DEFAULT GETDATE()	Upload timestamp.

Relationships: Store 1—Many StoreDocument

Category

Why this entity exists: Store-specific product categorization. Each store manages its own category set.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Category identifier.
StoreId	INT, FK → Store.Id	Store that owns the category.
Name	NVARCHAR(100), NOT NULL	Category name.
Icon	NVARCHAR(10), NULL	Small icon or emoji.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Store 1—Many Category; Category 1—Many Product

Important note: Recommended UNIQUE: (StoreId, Name).

Product

Why this entity exists: Store inventory item that can be listed, filtered, added to cart, and ordered.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Product identifier.
StoreId	INT, FK → Store.Id	Owner store.
CategoryId	INT, FK → Category.Id, NULL	Optional category.
Name	NVARCHAR(200), NOT NULL	Product name.
Description	NVARCHAR(2000), NULL	Product description.
Price	DECIMAL(10,2), NOT NULL	Base price.

SalePrice	DECIMAL(10,2), NULL	Discount price when applicable.
StockQuantity	INT, DEFAULT 0	Current stock count.
PetType	NVARCHAR(100), NULL	Target pet types stored as CSV.
ImageUrl	NVARCHAR(500), NULL	Product image.
IsVisible	BIT, DEFAULT 1	Visibility toggle.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Store 1—Many Product; Category 1—Many Product; Product 1—Many OrderItem and CartItem

Important note: Recommended CHECK: Price >= 0, SalePrice <= Price, StockQuantity >= 0.

Order

Why this entity exists: Store purchase order header. Captures recipient information, payment, and delivery metadata.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Order identifier.
UserId	INT, FK → Users.Id	Customer who placed the order.
StoreId	INT, FK → Store.Id	Store receiving the order.
OrderCode	NVARCHAR(30), NULL, UNIQUE	Trackable order code.
Status	NVARCHAR(20), DEFAULT 'Pending'	Pending, Confirmed, Preparing, Shipped, Delivered, or Cancelled.
PaymentMethod	NVARCHAR(20), NULL	Cash, Card, or Wallet.
PaymentStatus	NVARCHAR(20), DEFAULT 'Pending'	Pending, Paid, Failed, or Refunded.
RecipientName	NVARCHAR(100), NULL	Delivery recipient name.
RecipientPhone	NVARCHAR(20), NULL	Delivery recipient phone.
DeliveryAddress	NVARCHAR(300), NULL	Delivery address.
DeliveryFee	DECIMAL(10,2), NULL	Delivery fee.
Notes	NVARCHAR(500), NULL	Customer notes.
CancellationReason	NVARCHAR(300), NULL	Reason if cancelled.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—Many Order; Store 1—Many Order; Order 1—Many OrderItem

OrderItem

Why this entity exists: Immutable line items belonging to one order. Preserves pricing and naming snapshots at purchase time.

Attribute	Type	Note / Why needed

Id	INT, PK, IDENTITY	Order-item identifier.
OrderId	INT, FK → Order.Id	Parent order.
ProductId	INT, FK → Product.Id	Referenced product.
ProductNameSnapshot	NVARCHAR(200), NOT NULL	Product name at purchase time.
Quantity	INT, NOT NULL	Ordered quantity.
UnitPrice	DECIMAL(10,2), NOT NULL	Regular unit price at order time.
SalePrice	DECIMAL(10,2), NULL	Discounted unit price at order time if used.

Relationships: Order 1—Many OrderItem; Product 1—Many OrderItem

Important note: Recommended CHECK: Quantity > 0 and UnitPrice >= 0.

Cart

Why this entity exists: Pre-checkout shopping cart. Each user has one cart.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Cart identifier.
UserId	INT, FK → Users.Id, UNIQUE	Each user has one cart.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—1 Cart; Cart 1—Many CartItem

CartItem

Why this entity exists: Current selected products in a cart before checkout.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Cart-item identifier.
CartId	INT, FK → Cart.Id	Parent cart.
ProductId	INT, FK → Product.Id	Selected product.
Quantity	INT, NOT NULL	Selected quantity.

Relationships: Cart 1—Many CartItem; Product 1—Many CartItem

Important note: Recommended UNIQUE: (CartId, ProductId).

Important note: Recommended CHECK: Quantity > 0.

SubscriptionPlan

Why this entity exists: Master plan catalog for stores and clinics.

Attribute	Type	Note / Why needed

Id	INT, PK, IDENTITY	Subscription plan identifier.
Name	NVARCHAR(50), NOT NULL	Free, Basic, Professional, or Enterprise.
Price	DECIMAL(10,2), NOT NULL	Plan price.
DurationInDays	INT, NOT NULL	Plan duration in days.
MaxProducts	INT, NULL	Store product limit if applicable.
MaxDoctors	INT, NULL	Clinic doctor limit if applicable.
MaxAppointments	INT, NULL	Clinic appointment limit if applicable.
CommissionRate	DECIMAL(5,2), NULL	Optional commission percentage.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: SubscriptionPlan 1—Many Subscription

Important note: Recommended CHECK: CommissionRate BETWEEN 0 AND 100 when not null.

Subscription

Why this entity exists: Transactional subscription record for a clinic or store. This is now the single source of subscription truth.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Subscription identifier.
PlanId	INT, FK → SubscriptionPlan.Id	Chosen plan.
EntityType	NVARCHAR(20), NOT NULL	Clinic or Store.
EntityId	INT, NOT NULL	ID of the subscribed clinic or store.
StartDate	DATETIME2, NOT NULL	Subscription start.
EndDate	DATETIME2, NOT NULL	Subscription end.
AmountPaid	DECIMAL(10,2), NULL	Amount paid for this subscription.
PaymentStatus	NVARCHAR(20), DEFAULT 'Pending'	Pending, Paid, Failed, or Refunded.
Status	NVARCHAR(20), DEFAULT 'Active'	Active, Expired, or Cancelled.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: SubscriptionPlan 1—Many Subscription

Important note: Polymorphic reference is accepted intentionally for EntityType + EntityId.

Complaint

Why this entity exists: Complaint header table. Supports three user-facing complaint paths: clinic, store, or system.

Attribute	Type	Note / Why needed

Id	INT, PK, IDENTITY	Complaint identifier.
UserId	INT, FK → Users.Id	User who submitted the complaint.
TargetType	NVARCHAR(20), NOT NULL	Clinic, Store, or System.
TargetId	INT, NULL	Target clinic/store ID; null for system complaints.
Title	NVARCHAR(200), NOT NULL	Complaint title.
Description	NVARCHAR(1000), NOT NULL	Complaint details.
Status	NVARCHAR(20), DEFAULT 'Open'	Open, InProgress, Resolved, or Rejected.
Priority	NVARCHAR(20), DEFAULT 'Medium'	Low, Medium, or High.
ResolvedAt	DATETIME2, NULL	When the complaint was resolved.
ResolvedByAdminId	INT, FK → Users.Id, NULL	Admin who finalized the complaint.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—Many Complaint; Complaint 1—Many ComplaintReply and ComplaintReceiver

Important note: Business rule: system complaints have TargetId = NULL; clinic/store complaints require TargetId.

ComplaintReceiver

Why this entity exists: Routing table that sends a complaint to one or more admin viewers. Needed because clinic/store complaints now go to entity admin plus SuperAdmin.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Complaint-receiver identifier.
ComplaintId	INT, FK → Complaint.Id	Complaint being routed.
ReceiverUserId	INT, FK → Users.Id	Admin user who should see or process the complaint.
ReceiverRole	NVARCHAR(30), NOT NULL	ClinicAdmin, StoreAdmin, or SuperAdmin.
IsRead	BIT, DEFAULT 0	Whether this receiver opened the complaint.
ReadAt	DATETIME2, NULL	When this receiver opened it.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Routing timestamp.

Relationships: Complaint 1—Many ComplaintReceiver; Users 1—Many ComplaintReceiver

Important note: Clinic complaint: create rows for clinic admin and SuperAdmin.

Important note: Store complaint: create rows for store admin and SuperAdmin.

Important note: System complaint: create a row for SuperAdmin only.

ComplaintReply

Why this entity exists: Reply messages written by admins inside a complaint thread.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Reply identifier.
ComplaintId	INT, FK → Complaint.Id	Parent complaint.
AdminId	INT, FK → Users.Id	Admin who wrote the reply.
Message	NVARCHAR(1000), NOT NULL	Reply text.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Reply timestamp.

Relationships: Complaint 1—Many ComplaintReply

LostPetReport

Why this entity exists: Lost-pet report header. Supports both registered pets and manual snapshot data.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Lost report identifier.
UserId	INT, FK → Users.Id	User who created the report.
PetId	INT, FK → Pet.Id, NULL	Linked pet if already registered.
PetName	NVARCHAR(100), NULL	Snapshot pet name.
AnimalType	NVARCHAR(50), NULL	Pet type.
Breed	NVARCHAR(100), NULL	Breed snapshot.
Color	NVARCHAR(100), NULL	Color snapshot.
LastSeenLocation	NVARCHAR(300), NOT NULL	Last seen text location.
City	NVARCHAR(100), NOT NULL	City.
Area	NVARCHAR(100), NULL	Area.
Latitude	DECIMAL(9,6), NULL	Map latitude.
Longitude	DECIMAL(9,6), NULL	Map longitude.
DateLost	DATE, NOT NULL	Lost date.
TimeLost	TIME, NULL	Lost time.
Description	NVARCHAR(1000), NULL	Additional details.
SpecialMarks	NVARCHAR(500), NULL	Unique identifying marks.
MicrochipId	NVARCHAR(50), NULL	Optional microchip reference.
SecretQuestion	NVARCHAR(300), NOT NULL	Verification question.

SecretAnswerHash	NVARCHAR(500), NOT NULL	Hashed answer for security.
RewardAmount	DECIMAL(10,2), NULL	Reward value if offered.
RewardStatus	NVARCHAR(20), NULL	Frozen or Released.
Status	NVARCHAR(20), DEFAULT 'Active'	Active, Claimed, Found, or Expired.
ExpiryDate	DATETIME2, NULL	Report expiry.
ReportCode	NVARCHAR(20), NULL	Trackable report code.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—Many LostPetReport; LostPetReport 1—Many LostPetImage and FoundPetClaim

Important note: RewardAmount can also drive frozen-amount calculations for wallet availability.

LostPetImage

Why this entity exists: Images attached to a lost-pet report.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Image identifier.
ReportId	INT, FK → LostPetReport.Id	Parent lost-pet report.
ImageUrl	NVARCHAR(500), NOT NULL	Stored image path or URL.

Relationships: LostPetReport 1—Many LostPetImage

FoundPetClaim

Why this entity exists: Claim submitted by a finder against a lost-pet report.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Claim identifier.
LostReportId	INT, FK → LostPetReport.Id	Target lost-pet report.
FinderId	INT, FK → Users.Id	Finder user.
FoundLocation	NVARCHAR(300), NULL	Found-location text.
City	NVARCHAR(100), NULL	Found city.
Area	NVARCHAR(100), NULL	Found area.
Latitude	DECIMAL(9,6), NULL	Map latitude.
Longitude	DECIMAL(9,6), NULL	Map longitude.
FoundDate	DATE, NOT NULL	Found date.
FoundTime	TIME, NULL	Found time.
Description	NVARCHAR(1000), NULL	Finder notes.

AnimalType	NVARCHAR(50), NULL	Animal type snapshot.
Color	NVARCHAR(100), NULL	Color snapshot.
SpecialMarks	NVARCHAR(500), NULL	Observed marks.
MatchScore	DECIMAL(5,2), NULL	Calculated matching score.
AttemptsCount	INT, DEFAULT 0	Number of verification attempts used.
FinderCode	NVARCHAR(20), NULL	Finder-side verification code.
OwnerCode	NVARCHAR(20), NULL	Owner-side verification code.
CodesExchanged	BIT, DEFAULT 0	Whether codes have been exchanged.
Status	NVARCHAR(20), DEFAULT 'Pending'	Pending, Verified, Rejected, or Locked.
ExpiresAt	DATETIME2, NULL	Claim expiry.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: LostPetReport 1—Many FoundPetClaim; Users 1—Many FoundPetClaim; FoundPetClaim 1—Many FoundPetImage and ClaimAttemptLog

FoundPetImage

Why this entity exists: Images attached to a found-pet claim.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Image identifier.
ClaimId	INT, FK → FoundPetClaim.Id	Parent claim.
ImageUrl	NVARCHAR(500), NOT NULL	Stored image path or URL.

Relationships: FoundPetClaim 1—Many FoundPetImage

ClaimAttemptLog

Why this entity exists: Audit trail of secret-answer attempts during lost/found verification.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Attempt identifier.
ClaimId	INT, FK → FoundPetClaim.Id	Parent claim.
AttemptValue	NVARCHAR(300), NOT NULL	Submitted answer/value.
IsCorrect	BIT, NOT NULL	Whether the attempt was correct.
IPAddress	NVARCHAR(50), NULL	Optional IP address for abuse tracking.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Attempt timestamp.

Relationships: FoundPetClaim 1—Many ClaimAttemptLog

DangerousAnimalReport

Why this entity exists: Community report for dangerous, stray, injured, or problematic animals.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Report identifier.
ReporterId	INT, FK → Users.Id	User who reported the issue.
AnimalType	NVARCHAR(50), NOT NULL	Animal type.
ProblemType	NVARCHAR(100), NOT NULL	Aggressive, Injured, Sick, Stray, Attack, Noise, etc.
Severity	NVARCHAR(20), NOT NULL	Low, Medium, or High.
Description	NVARCHAR(1000), NOT NULL	Problem description.
City	NVARCHAR(100), NOT NULL	City.
Area	NVARCHAR(100), NULL	Area.
LocationText	NVARCHAR(300), NULL	Human-readable location.
Latitude	DECIMAL(9,6), NULL	Map latitude.
Longitude	DECIMAL(9,6), NULL	Map longitude.
Status	NVARCHAR(20), DEFAULT 'Open'	Open, Processing, Resolved, or Fake.
ReportCode	NVARCHAR(20), NULL	Trackable report code.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—Many DangerousAnimalReport; DangerousAnimalReport 1—Many DangerousAnimalImage

DangerousAnimalImage

Why this entity exists: Images attached to a dangerous-animal report.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Image identifier.
ReportId	INT, FK → DangerousAnimalReport.Id	Parent report.
ImageUrl	NVARCHAR(500), NOT NULL	Stored image path or URL.

Relationships: DangerousAnimalReport 1—Many DangerousAnimalImage

AdoptionPost

Why this entity exists: Adoption listing that may be posted by a user, clinic, or store.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Adoption post identifier.
PostedByType	NVARCHAR(20), NOT NULL	User, Clinic, or Store.

PostedById	INT, NOT NULL	ID of the posting entity.
PetId	INT, FK → Pet.Id, NULL	Linked pet if registered.
PetName	NVARCHAR(100), NULL	Snapshot name.
AnimalType	NVARCHAR(50), NULL	Animal type.
Breed	NVARCHAR(100), NULL	Breed.
Color	NVARCHAR(100), NULL	Color.
Age	INT, NULL	Approximate age when no registered pet age is available.
Gender	NVARCHAR(10), NULL	Gender snapshot.
EnergyLevel	NVARCHAR(20), NULL	Behavior energy level.
GoodWithChildren	BIT, NULL	Compatibility with children.
GoodWithPets	BIT, NULL	Compatibility with other pets.
HealthStatus	NVARCHAR(200), NULL	Health summary.
IsVaccinated	BIT, NULL	Vaccination indicator.
IsNeutered	BIT, NULL	Neutering indicator.
City	NVARCHAR(100), NULL	Location city.
Area	NVARCHAR(100), NULL	Location area.
ContactName	NVARCHAR(100), NULL	Who should be contacted about the post.
ContactPhone	NVARCHAR(20), NULL	Contact phone.
Description	NVARCHAR(1000), NULL	Listing description.
ReasonForAdoption	NVARCHAR(500), NULL	Reason for rehoming or posting.
Status	NVARCHAR(20), DEFAULT 'Pending'	Pending, Approved, or Rejected.
AdoptionStatus	NVARCHAR(20), DEFAULT 'Available'	Available, Reserved, or Adopted.
PostCode	NVARCHAR(20), NULL	Trackable post code.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: AdoptionPost 1—Many AdoptionImage

AdoptionImage

Why this entity exists: Images attached to an adoption post.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Image identifier.
PostId	INT, FK → AdoptionPost.Id	Parent adoption post.

ImageUrl	NVARCHAR(500), NOT NULL	Stored image path or URL.

Relationships: AdoptionPost 1—Many AdoptionImage

Donation

Why this entity exists: Donation transaction table for money or physical goods.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Donation identifier.
DonorId	INT, FK → Users.Id	Donating user.
DonationType	NVARCHAR(20), NOT NULL	Money, Food, Medicine, or Supplies.
Amount	DECIMAL(10,2), NULL	Monetary amount for cash donations.
Quantity	INT, NULL	Quantity for item donations.
Unit	NVARCHAR(50), NULL	Bag, Box, Piece, etc.
Description	NVARCHAR(500), NULL	Donation note.
ProofImageUrl	NVARCHAR(500), NULL	Proof image.
TargetType	NVARCHAR(20), NULL	LostPet, Report, Pet, or General.
TargetId	INT, NULL	Target entity ID.
ReferenceCode	NVARCHAR(50), NULL	Trackable reference.
Status	NVARCHAR(20), DEFAULT 'Pending'	Pending, Approved, or Distributed.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—Many Donation

PlatformLaw

Why this entity exists: Admin-managed laws and policy content shown in the platform.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Law identifier.
Title	NVARCHAR(200), NOT NULL	Law/article title.
Category	NVARCHAR(50), NOT NULL	Travel, Adoption, Protection, Vaccination, Breeding, Stray, etc.
Content	NVARCHAR(MAX), NOT NULL	Full content body.
Source	NVARCHAR(300), NULL	Legal source or citation.
IssuedDate	DATE, NULL	Official issue date.
LastUpdated	DATETIME2, DEFAULT GETDATE()	Last content update timestamp.
CreatedBy	INT, FK → Users.Id	Admin who created the entry.

Relationships: Users 1—Many PlatformLaw entries created by admins

Notification

Why this entity exists: User notification table for platform events and reminders.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Notification identifier.
UserId	INT, FK → Users.Id	Target user.
Type	NVARCHAR(50), NOT NULL	Booking, Order, LostPet, Adoption, Complaint, System, Donation, etc.
Title	NVARCHAR(200), NOT NULL	Short notification title.
Message	NVARCHAR(500), NOT NULL	Notification message body.
IsRead	BIT, DEFAULT 0	Whether the user has read the notification.
ReadAt	DATETIME2, NULL	When the notification was read.
RelatedEntityType	NVARCHAR(50), NULL	Optional linked entity type.
RelatedEntityId	INT, NULL	Optional linked entity ID.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—Many Notification

Review

Why this entity exists: Polymorphic review table for rating clinics, doctors, and stores.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Review identifier.
UserId	INT, FK → Users.Id	User who wrote the review.
TargetType	NVARCHAR(20), NOT NULL	Clinic, Doctor, or Store.
TargetId	INT, NOT NULL	ID of the reviewed target.
Rating	INT, NOT NULL	Numeric rating from 1 to 5.
Comment	NVARCHAR(1000), NULL	Optional written review.
Status	NVARCHAR(20), DEFAULT 'Visible'	Visible or Hidden.
UpdatedAt	DATETIME2, NULL	Last review edit time.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—Many Review

Important note: Recommended UNIQUE: (UserId, TargetType, TargetId).

Important note: Recommended CHECK: Rating BETWEEN 1 AND 5.

ReportActionLog

Why this entity exists: Admin audit log for moderation actions across complaints and report-like entities.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Log identifier.
EntityType	NVARCHAR(50), NOT NULL	Complaint, DangerousAnimal, LostPet, Adoption, etc.
EntityId	INT, NOT NULL	ID of the moderated entity.
ActionType	NVARCHAR(50), NOT NULL	Approved, Rejected, Resolved, Hidden, etc.
StatusAfterAction	NVARCHAR(20), NULL	Resulting status after the action.
Note	NVARCHAR(500), NULL	Optional admin note.
AdminId	INT, FK → Users.Id	Admin who performed the action.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Action timestamp.

Relationships: Users 1—Many ReportActionLog entries written by admins

AdminMapMarker (Optional)

Why this entity exists: Optional admin-only map marker layer for custom manual markers beyond the core operational tables.

Attribute	Type	Note / Why needed
Id	INT, PK, IDENTITY	Marker identifier.
MarkerType	NVARCHAR(50), NOT NULL	Clinic, Store, LostPet, DangerousAnimal, Manual, etc.
ReferenceType	NVARCHAR(50), NULL	Optional linked entity type.
ReferenceId	INT, NULL	Optional linked entity ID.
Latitude	DECIMAL(9,6), NOT NULL	Marker latitude.
Longitude	DECIMAL(9,6), NOT NULL	Marker longitude.
Status	NVARCHAR(20), DEFAULT 'Active'	Active, Hidden, or Resolved.
CreatedByAdminId	INT, FK → Users.Id	Admin who placed the marker.
CreatedAt	DATETIME2, DEFAULT GETDATE()	Creation timestamp.

Relationships: Users 1—Many AdminMapMarker entries created by admins

Relationship map and integrity rules

• Users 1—1 Wallet; Wallet 1—Many WalletTransaction.

• Users 1—Many Pet.

• Users 1—1 Clinic through Clinic.OwnerUserId (UNIQUE).

• Users 1—1 Store through Store.OwnerUserId (UNIQUE).

• Users 1—1 Doctor through Doctor.UserId (UNIQUE).

• Clinic 1—Many Doctor, ClinicSchedule, ClinicService, ClinicDocument, Appointment, and Complaint.

• Doctor 1—Many DoctorService, DoctorDocument, and Appointment.

• Users 1—Many Appointment; Pet 1—Many Appointment.

• Store 1—Many Category, Product, StoreDocument, Order, and Complaint.

• Category 1—Many Product.

• Users 1—1 Cart; Cart 1—Many CartItem; Product 1—Many CartItem.

• Order 1—Many OrderItem; Product 1—Many OrderItem.

• SubscriptionPlan 1—Many Subscription; Subscription polymorphically targets Clinic or Store.

• Users 1—Many Complaint; Complaint 1—Many ComplaintReply and ComplaintReceiver.

• Users 1—Many LostPetReport; LostPetReport 1—Many LostPetImage and FoundPetClaim.

• FoundPetClaim 1—Many FoundPetImage and ClaimAttemptLog.

• Users 1—Many DangerousAnimalReport; DangerousAnimalReport 1—Many DangerousAnimalImage.

• Pet 1—Many LostPetReport and AdoptionPost (optional links).

• AdoptionPost 1—Many AdoptionImage.

• Users 1—Many Donation, Notification, Review, ReportActionLog, and optional AdminMapMarker.

• Review polymorphically targets Clinic, Doctor, or Store.

• Appointment deposit activity is auditable through WalletTransaction using ReferenceType = 'Appointment'.

Key constraints and implementation notes

• Use DATETIME2 instead of DATETIME across the schema.

• Recommended CHECK constraints include: WarningCount >= 0; Balance >= 0; Amount > 0; Price >= 0; SalePrice <= Price; StockQuantity >= 0; Quantity > 0; Rating BETWEEN 1 AND 5; DayOfWeek BETWEEN 0 AND 6; ExperienceYears >= 0; AppointmentDuration > 0; StartTime < EndTime; OpenTime < CloseTime; BookingDeposit >= 0.

• Recommended UNIQUE constraints include: Clinic.OwnerUserId; Store.OwnerUserId; Wallet.UserId; Cart.UserId; Doctor.UserId; Category(StoreId, Name); ClinicSchedule(ClinicId, DayOfWeek); CartItem(CartId, ProductId); Review(UserId, TargetType, TargetId).

• Clinic and Store no longer contain subscription plan/expiry fields because all subscription logic now lives in Subscription and SubscriptionPlan.

• Doctor.InviteCode was removed intentionally because clinics now create doctor accounts directly.

• Complaint routing now requires ComplaintReceiver so a single complaint can be seen by more than one admin receiver.

• Appointment deposits should be charged at booking and later refunded or forfeited according to attendance outcome.

• CSV fields are accepted deliberately for academic scope: Pet.Traits, Doctor.WorkingDays, Store.WorkingDays, Store.AcceptedPayments, and some Product.PetType values.

• Snapshot fields are intentional for historical correctness: OrderItem.ProductNameSnapshot, Appointment.ServiceName, LostPetReport pet snapshots, AdoptionPost pet snapshots.

• Wallet.Balance and cached ratings (Clinic/Doctor/Store) may be stored for quick display, but their source of truth remains query-based.

• Polymorphic references do not provide full FK enforcement at the DB level; validate them in application logic as well.

Computed data and query catalog

Policy: Calculated values should usually be generated from queries at runtime instead of being stored permanently. Cached values like Wallet.Balance or Clinic/Doctor/Store.Rating are allowed for fast display, but their source remains query-based.

#	Computed field	Source tables	Why needed
1	AverageClinicRating	Review	Compute visible average rating for a clinic instead of storing a permanent value.
2	AverageDoctorRating	Review	Compute visible average rating for a doctor.
3	AverageStoreRating	Review	Compute visible average rating for a store.
4	TotalReviews	Review	Count visible reviews for any reviewable target.
5	PetAge	Pet	Calculate pet age from birth date when needed.
6	CurrentWalletBalance	WalletTransaction	Recalculate the current wallet balance from completed transactions.
7	CartItemsCount	CartItem	Count the total quantity of items in a cart.
8	CartSubtotal	CartItem + Product	Calculate the cart subtotal from active product prices.
9	OrderSubtotal	OrderItem	Calculate an order subtotal without delivery fee.

10	OrderTotal	Order + OrderItem	Calculate the final total including delivery fee.
11	StoreProductsCount	Product	Count how many products belong to a store.
12	StoreOrdersCount	Order	Count all orders for a store.
13	OrdersByStatus	Order	Return order counts grouped by status for dashboard use.
14	DoctorAppointmentsCount	Appointment	Count all appointments assigned to a doctor.
15	TodayAppointmentsCount	Appointment	Count today's active appointments for a doctor.
16	ClinicDoctorsCount	Doctor	Count how many doctors belong to a clinic.
17	DoctorServicesCount	DoctorService	Count active services for one doctor.
18	ClinicServicesCount	ClinicService	Count active clinic services.
19	UserPetsCount	Pet	Count pets owned by one user.
20	ActiveLostReportsCount	LostPetReport	Count active lost-pet reports.
21	ClaimsCount	FoundPetClaim	Count submitted claims against a lost-pet report.
22	BestMatchScore	FoundPetClaim	Return the highest match score among claims for one lost report.
23	AttemptsUsed	ClaimAttemptLog	Count verification attempts used for a claim.
24	AvailableAdoptionsCount	AdoptionPost	Count approved adoption posts that are still available.
25	DonationsCount	Donation	Count all donation records.
26	TotalMoneyDonations	Donation	Sum approved/distributed monetary donations.
27	UnreadNotificationsCount	Notification	Count unread notifications for one user.
28	ComplaintsByStatus	Complaint	Return complaint counts grouped by status.
29	DangerousReportsBySeverity	DangerousAnimalReport	Return dangerous-animal reports grouped by severity.
30	UserComplaintInbox	Complaint + ComplaintReceiver	Return all complaints routed to a specific admin user.

31	IsFullyBooked	Appointment + Doctor	Check whether a doctor has reached the maximum appointment limit on a date.
32	FrozenAmount	LostPetReport	Calculate reward amounts still frozen for a user's active lost-pet reports.
33	PendingOrdersToday	Order	Count today's pending store orders for dashboard badges.
34	TopSellingProducts	OrderItem + Product	Return the top five best-selling products and their revenue for one store.
35	UserCartConversionRate	Cart + Order	Estimate the percentage of users with carts who eventually created orders for a store.
36	CalculatedMatchScore	FoundPetClaim + LostPetReport	Compute a matching score to help populate MatchScore during claim evaluation.
37	PetDonationStats	Donation	Summarize donation activity for a specific pet target.
38	AppointmentDepositState	Appointment	Return held/refunded/forfeited deposit information for one appointment.

Query details

1. AverageClinicRating

Source: Review

Why needed: Compute visible average rating for a clinic instead of storing a permanent value.

```
SELECT AVG(CAST(Rating AS DECIMAL(3,2))) AS AverageClinicRating
FROM Review
WHERE TargetType = 'Clinic' AND TargetId = @ClinicId AND Status = 'Visible';
```

2. AverageDoctorRating

Source: Review

Why needed: Compute visible average rating for a doctor.

```
SELECT AVG(CAST(Rating AS DECIMAL(3,2))) AS AverageDoctorRating
FROM Review
WHERE TargetType = 'Doctor' AND TargetId = @DoctorId AND Status = 'Visible';
```

3. AverageStoreRating

Source: Review

Why needed: Compute visible average rating for a store.

```
SELECT AVG(CAST(Rating AS DECIMAL(3,2))) AS AverageStoreRating
FROM Review
WHERE TargetType = 'Store' AND TargetId = @StoreId AND Status = 'Visible';
```

4. TotalReviews

Source: Review

Why needed: Count visible reviews for any reviewable target.

```
SELECT COUNT(*) AS TotalReviews
FROM Review
WHERE TargetType = @TargetType AND TargetId = @TargetId AND Status = 'Visible';
```

5. PetAge

Source: Pet

Why needed: Calculate pet age from birth date when needed.

```
SELECT DATEDIFF(YEAR, BirthDate, GETDATE()) AS PetAge
FROM Pet
WHERE Id = @PetId;
```

6. CurrentWalletBalance

Source: WalletTransaction

Why needed: Recalculate the current wallet balance from completed transactions.

```
SELECT SUM(
    CASE
        WHEN Type IN ('Deposit','Cashback','Reward','Refund') THEN Amount
        WHEN Type IN ('Withdraw','Payment','Fee') THEN -Amount
        ELSE 0
    END
) AS CurrentWalletBalance
FROM WalletTransaction
WHERE WalletId = @WalletId AND Status = 'Completed';
```

7. CartItemsCount

Source: CartItem

Why needed: Count the total quantity of items in a cart.

```
SELECT SUM(Quantity) AS CartItemsCount
FROM CartItem
WHERE CartId = @CartId;
```

8. CartSubtotal

Source: CartItem + Product

Why needed: Calculate the cart subtotal from active product prices.

```
SELECT SUM(ci.Quantity * ISNULL(p.SalePrice, p.Price)) AS CartSubtotal
FROM CartItem ci
JOIN Product p ON ci.ProductId = p.Id
WHERE ci.CartId = @CartId;
```

9. OrderSubtotal

Source: OrderItem

Why needed: Calculate an order subtotal without delivery fee.

```
SELECT SUM(Quantity * ISNULL(SalePrice, UnitPrice)) AS OrderSubtotal
FROM OrderItem
WHERE OrderId = @OrderId;
```

10. OrderTotal

Source: Order + OrderItem

Why needed: Calculate the final total including delivery fee.

```
SELECT SUM(oi.Quantity * ISNULL(oi.SalePrice, oi.UnitPrice)) + ISNULL(o.DeliveryFee, 0) AS OrderTotal
FROM [Order] o
JOIN OrderItem oi ON o.Id = oi.OrderId
WHERE o.Id = @OrderId
GROUP BY o.DeliveryFee;
```

11. StoreProductsCount

Source: Product

Why needed: Count how many products belong to a store.

```
SELECT COUNT(*) AS StoreProductsCount
FROM Product
WHERE StoreId = @StoreId;
```

12. StoreOrdersCount

Source: Order

Why needed: Count all orders for a store.

```
SELECT COUNT(*) AS StoreOrdersCount
FROM [Order]
WHERE StoreId = @StoreId;
```

13. OrdersByStatus

Source: Order

Why needed: Return order counts grouped by status for dashboard use.

```
SELECT Status, COUNT(*) AS Total
FROM [Order]
WHERE StoreId = @StoreId
GROUP BY Status;
```

14. DoctorAppointmentsCount

Source: Appointment

Why needed: Count all appointments assigned to a doctor.

```
SELECT COUNT(*) AS DoctorAppointmentsCount
FROM Appointment
WHERE DoctorId = @DoctorId;
```

15. TodayAppointmentsCount

Source: Appointment

Why needed: Count today's active appointments for a doctor.

```
SELECT COUNT(*) AS TodayAppointmentsCount
FROM Appointment
WHERE DoctorId = @DoctorId
  AND AppointmentDate = CAST(GETDATE() AS DATE)
  AND Status IN ('Pending','Confirmed','Completed');
```

16. ClinicDoctorsCount

Source: Doctor

Why needed: Count how many doctors belong to a clinic.

```
SELECT COUNT(*) AS ClinicDoctorsCount
FROM Doctor
WHERE ClinicId = @ClinicId;
```

17. DoctorServicesCount

Source: DoctorService

Why needed: Count active services for one doctor.

```
SELECT COUNT(*) AS DoctorServicesCount
FROM DoctorService
WHERE DoctorId = @DoctorId AND IsActive = 1;
```

18. ClinicServicesCount

Source: ClinicService

Why needed: Count active clinic services.

```
SELECT COUNT(*) AS ClinicServicesCount
FROM ClinicService
WHERE ClinicId = @ClinicId AND IsActive = 1;
```

19. UserPetsCount

Source: Pet

Why needed: Count pets owned by one user.

```
SELECT COUNT(*) AS UserPetsCount
FROM Pet
WHERE OwnerId = @UserId;
```

20. ActiveLostReportsCount

Source: LostPetReport

Why needed: Count active lost-pet reports.

```
SELECT COUNT(*) AS ActiveLostReportsCount
FROM LostPetReport
WHERE Status = 'Active';
```

21. ClaimsCount

Source: FoundPetClaim

Why needed: Count submitted claims against a lost-pet report.

```
SELECT COUNT(*) AS ClaimsCount
FROM FoundPetClaim
WHERE LostReportId = @LostReportId;
```

22. BestMatchScore

Source: FoundPetClaim

Why needed: Return the highest match score among claims for one lost report.

```
SELECT MAX(MatchScore) AS BestMatchScore
FROM FoundPetClaim
WHERE LostReportId = @LostReportId;
```

23. AttemptsUsed

Source: ClaimAttemptLog

Why needed: Count verification attempts used for a claim.

```
SELECT COUNT(*) AS AttemptsUsed
FROM ClaimAttemptLog
WHERE ClaimId = @ClaimId;
```

24. AvailableAdoptionsCount

Source: AdoptionPost

Why needed: Count approved adoption posts that are still available.

```
SELECT COUNT(*) AS AvailableAdoptionsCount
FROM AdoptionPost
WHERE AdoptionStatus = 'Available' AND Status = 'Approved';
```

25. DonationsCount

Source: Donation

Why needed: Count all donation records.

```
SELECT COUNT(*) AS DonationsCount
FROM Donation;
```

26. TotalMoneyDonations

Source: Donation

Why needed: Sum approved/distributed monetary donations.

```
SELECT SUM(Amount) AS TotalMoneyDonations
FROM Donation
WHERE DonationType = 'Money' AND Status IN ('Approved','Distributed');
```

27. UnreadNotificationsCount

Source: Notification

Why needed: Count unread notifications for one user.

```
SELECT COUNT(*) AS UnreadNotificationsCount
FROM Notification
WHERE UserId = @UserId AND IsRead = 0;
```

28. ComplaintsByStatus

Source: Complaint

Why needed: Return complaint counts grouped by status.

```
SELECT Status, COUNT(*) AS Total
FROM Complaint
GROUP BY Status;
```

29. DangerousReportsBySeverity

Source: DangerousAnimalReport

Why needed: Return dangerous-animal reports grouped by severity.

```
SELECT Severity, COUNT(*) AS Total
FROM DangerousAnimalReport
GROUP BY Severity;
```

30. UserComplaintInbox

Source: Complaint + ComplaintReceiver

Why needed: Return all complaints routed to a specific admin user.

```
SELECT c.*
FROM Complaint c
JOIN ComplaintReceiver cr ON c.Id = cr.ComplaintId
WHERE cr.ReceiverUserId = @AdminUserId
ORDER BY c.CreatedAt DESC;
```

31. IsFullyBooked

Source: Appointment + Doctor

Why needed: Check whether a doctor has reached the maximum appointment limit on a date.

```
SELECT CASE
    WHEN COUNT(*) >= d.MaxDailyAppointments THEN 1
    ELSE 0
END AS IsFullyBooked
FROM Appointment a
JOIN Doctor d ON a.DoctorId = d.Id
WHERE a.DoctorId = @DoctorId
  AND a.AppointmentDate = @Date
  AND a.Status IN ('Pending','Confirmed')
GROUP BY d.MaxDailyAppointments;
```

32. FrozenAmount

Source: LostPetReport

Why needed: Calculate reward amounts still frozen for a user’s active lost-pet reports.

```
SELECT SUM(RewardAmount) AS FrozenAmount
FROM LostPetReport
WHERE UserId = @UserId
  AND RewardStatus = 'Frozen'
  AND Status = 'Active';
```

33. PendingOrdersToday

Source: Order

Why needed: Count today's pending store orders for dashboard badges.

```
SELECT COUNT(*) AS PendingOrdersToday
FROM [Order]
WHERE StoreId = @StoreId
  AND Status = 'Pending'
  AND CAST(CreatedAt AS DATE) = CAST(GETDATE() AS DATE);
```

34. TopSellingProducts

Source: OrderItem + Product

Why needed: Return the top five best-selling products and their revenue for one store.

```
SELECT TOP 5
  p.Id, p.Name,
  SUM(oi.Quantity) AS TotalSold,
  SUM(oi.Quantity * ISNULL(oi.SalePrice, oi.UnitPrice)) AS TotalRevenue
FROM OrderItem oi
JOIN Product p ON oi.ProductId = p.Id
WHERE p.StoreId = @StoreId
GROUP BY p.Id, p.Name
ORDER BY TotalSold DESC;
```

35. UserCartConversionRate

Source: Cart + Order

Why needed: Estimate the percentage of users with carts who eventually created orders for a store.

```
SELECT
  COUNT(DISTINCT c.UserId) AS UsersWithCart,
  COUNT(DISTINCT o.UserId) AS UsersWhoBought,
  CAST(COUNT(DISTINCT o.UserId) AS FLOAT) /
  NULLIF(COUNT(DISTINCT c.UserId), 0) * 100 AS ConversionRate
FROM Cart c
LEFT JOIN [Order] o ON c.UserId = o.UserId
  AND o.StoreId = @StoreId;
```

36. CalculatedMatchScore

Source: FoundPetClaim + LostPetReport

Why needed: Compute a matching score to help populate MatchScore during claim evaluation.

```
SELECT
  f.Id,
  (
    CASE WHEN f.AnimalType = l.AnimalType THEN 40 ELSE 0 END +
    CASE WHEN f.Color = l.Color THEN 30 ELSE 0 END +
    CASE WHEN f.City = l.City THEN 20 ELSE 0 END +
    CASE WHEN f.Area = l.Area THEN 10 ELSE 0 END
```

```
  ) AS CalculatedMatchScore
FROM FoundPetClaim f
JOIN LostPetReport l ON f.LostReportId = l.Id
WHERE f.Id = @ClaimId;
```

37. PetDonationStats

Source: Donation

Why needed: Summarize donation activity for a specific pet target.

```
SELECT
  TargetId,
  COUNT(*) AS DonationsCount,
  SUM(CASE WHEN DonationType = 'Money' THEN Amount ELSE 0 END) AS MoneyTotal,
  SUM(CASE WHEN DonationType <> 'Money' THEN Quantity ELSE 0 END) AS ItemsTotal
FROM Donation
WHERE TargetType = 'Pet'
  AND TargetId = @PetId
  AND Status IN ('Approved','Distributed')
GROUP BY TargetId;
```

38. AppointmentDepositState

Source: Appointment

Why needed: Return held/refunded/forfeited deposit information for one appointment.

```
SELECT Id, BookingDeposit, DepositStatus, Status
FROM Appointment
WHERE Id = @AppointmentId;
```

Code SQL Server

polymorphic relation:

- This means we don't put FK directly in SQL Server because the table can be associated with more than one entity type, and this needs to be checked backend in ASP.NET Core + Dapper

- Subscription
- WalletTransaction
- Review
- AdoptionPost
- Donation
- Notification
- AdminMapMarker
- ReportActionLog

Database

User :

function: The primary table for all accounts in the system.

linked to: UserRole, Wallet, Cart, Pet, Appointment, Complaint, Donation, Notification, Review, and can also be the owner of a Clinic, Store, or Doctor account.

```
CREATE TABLE dbo.[Users]
(
    [Id]                INT IDENTITY(1,1) NOT NULL,
    [Email]             NVARCHAR(150) NOT NULL,
    [PasswordHash]      NVARCHAR(255) NOT NULL,
    [Phone]             NVARCHAR(20) NOT NULL,
    [IsPhoneVerified]   BIT NOT NULL CONSTRAINT DF_Users_IsPhoneVerified DEFAULT (0),
    [IsEmailVerified]   BIT NOT NULL CONSTRAINT DF_Users_IsEmailVerified DEFAULT (0),
```

```
    [FullName]          NVARCHAR(100) NOT NULL,
    [City]              NVARCHAR(100) NOT NULL,
    [ProfileImage]      NVARCHAR(500) NULL,
    [Area]              NVARCHAR(100) NULL,
    [AddressDetail]     NVARCHAR(300) NULL,
    [Latitude]          DECIMAL(9,6) NULL,
    [Longitude]         DECIMAL(9,6) NULL,
    [WhatsAppPhone]     NVARCHAR(20) NULL,
    [BirthDate]         DATE NULL,
    [Gender]            NVARCHAR(20) NULL,
    [Bio]               NVARCHAR(500) NULL,
    [AccountStatus]     NVARCHAR(20) NOT NULL CONSTRAINT DF_Users_AccountStatus DEFAULT
(N'Active'),
    [WarningCount]      INT NOT NULL CONSTRAINT DF_Users_WarningCount DEFAULT (0),
    [SuspensionReason]  NVARCHAR(500) NULL,
    [CreatedAt]         DATETIME2 NOT NULL CONSTRAINT DF_Users_CreatedAt DEFAULT (GETDATE()),
    [LastActiveAt]      DATETIME2 NULL,
    CONSTRAINT PK_Users PRIMARY KEY CLUSTERED ([Id]),
    CONSTRAINT UQ_Users_Email UNIQUE ([Email]),
    CONSTRAINT CK_Users_WarningCount CHECK ([WarningCount] >= 0),
    CONSTRAINT CK_Users_AccountStatus CHECK ([AccountStatus] IN (N'Active', N'Suspended',
N'Banned'))
);
```

Role :

function: To schedule roles such as PetOwner, ClinicOwner, Doctor, StoreOwner, and SuperAdmin.

linked to: UserRole.

```
CREATE TABLE dbo.[Role]
(
    [Id]        INT IDENTITY(1,1) NOT NULL,
    [Name]      NVARCHAR(50) NOT NULL,
    CONSTRAINT PK_Role PRIMARY KEY CLUSTERED ([Id]),
    CONSTRAINT UQ_Role_Name UNIQUE ([Name])
);
```

UserRole :

function: To link the user to the role.

It is associated with: Users, Roles.

Note: The primary key is composed of UserId + RoleId.

```
CREATE TABLE dbo.[UserRole] (
```

```
[UserId]            INT NOT NULL,
[RoleId]             INT NOT NULL,
[AssignedAt]     DATETIME2 NOT NULL CONSTRAINT DF_UserRole_AssignedAt DEFAULT (GETDATE()),
CONSTRAINT     PK_UserRole PRIMARY KEY CLUSTERED ([UserId], [RoleId]),
CONSTRAINT  FK_UserRole_User FOREIGN KEY ([UserId]) REFERENCES dbo.Users,
CONSTRAINT  FK_UserRole_Role FOREIGN KEY ([RoleId]) REFERENCES dbo.Role
);
```

Wallet :

Function: One wallet per user.

Connected with: Users, WalletTransaction.

Note: Each user has only one wallet.

```
CREATE TABLE dbo.[Wallet] (
[Id]       INT IDENTITY(1,1) NOT NULL,
[UserId]       INT NOT NULL,
[Balance]        DECIMAL(10,2) NOT NULL CONSTRAINT DF_Wallet_Balance DEFAULT (0.00),
[Status]       NVARCHAR(20) NOT NULL CONSTRAINT DF_Wallet_Status DEFAULT (N'Active'),
[CreatedAt]       DATETIME2 NOT NULL CONSTRAINT DF_Wallet_CreatedAt DEFAULT (GETDATE()),
[UpdatedAt]      DATETIME2 NULL, CONSTRAINT PK_Wallet PRIMARY KEY CLUSTERED ([Id]),
CONSTRAINT UQ_Wallet_UserId UNIQUE ([UserId]),
CONSTRAINT FK_Wallet_User FOREIGN KEY ([UserId]) REFERENCES dbo.Users,
CONSTRAINT CK_Wallet_Balance CHECK ([Balance] >= 0),
CONSTRAINT CK_Wallet_Status CHECK ([Status] IN (N'Active', N'Suspended')) );
```

WalletTransaction:

Its function: To record all wallet transactions.

It is associated with: Wallet.

Important note: ReferenceType + ReferenceId are polymorphic relationships, meaning verification is done via backend, not direct FK.

```
CREATE TABLE dbo.[WalletTransaction] (
[Id]              INT IDENTITY(1,1) NOT NULL,
[WalletId]              INT NOT NULL,
```

```
[TransactionCode]         NVARCHAR(50) NULL,
[Type]          NVARCHAR(30) NOT NULL,
[Amount]           DECIMAL(10,2) NOT NULL,
[PaymentMethod]          NVARCHAR(20) NULL,
[Description]            NVARCHAR(300) NULL,
[ReferenceType]               NVARCHAR(50) NULL,
[ReferenceId]           INT NULL,
[Status]             NVARCHAR(20) NOT NULL DEFAULT (N'Completed'),
[CreatedAt]            DATETIME2 NOT NULL DEFAULT (GETDATE()),
CONSTRAINT  PK_WalletTransaction PRIMARY KEY ([Id]),
CONSTRAINT UQ_WalletTransaction_TransactionCode UNIQUE ([TransactionCode]),
CONSTRAINT FK_WalletTransaction_Wallet FOREIGN KEY ([WalletId]) REFERENCES dbo.Wallet,
CONSTRAINT CK_WalletTransaction_Amount CHECK ([Amount] > 0),
CONSTRAINT CK_WalletTransaction_Status CHECK ([Status] IN (N'Completed', N'Pending',
N'Failed'))
);
```

Pet:

Its function: User-related pet data.

It integrates with: Users, Appointment, LostPetReport, AdoptionPost.

note: Unique Index works on MicrochipId if Not Null

```
CREATE TABLE dbo.[Pet]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [OwnerId] INT NOT NULL,
 [Name] NVARCHAR(100) NOT NULL,
 [Species] NVARCHAR(50) NOT NULL,
 [Breed] NVARCHAR(100) NULL,
 [Gender] NVARCHAR(10) NOT NULL,
 [BirthDate] DATE NULL,
 [Color] NVARCHAR(100) NULL,
 [MicrochipId] NVARCHAR(50) NULL,
 [Weight] DECIMAL(5,2) NULL,
 [VaccinationStatus] NVARCHAR(20) NULL,
 [IsNeutered] BIT NOT NULL DEFAULT (0),
 [MedicalNotes] NVARCHAR(1000) NULL,
 [Traits] NVARCHAR(500) NULL,
 [EnergyLevel] NVARCHAR(20) NULL,
 [GoodWithChildren] BIT NULL,
 [GoodWithPets] BIT NULL,
 [IsTrained] BIT NOT NULL DEFAULT (0),
 [ImageUrl] NVARCHAR(500) NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_Pet PRIMARY KEY ([Id]),
 CONSTRAINT FK_Pet_User FOREIGN KEY ([OwnerId]) REFERENCES dbo.[Users]([Id]),
 CONSTRAINT CK_Pet_Weight CHECK ([Weight] IS NULL OR [Weight] >= 0)

);
```

Clinic:

Its function: Clinic data.

It is associated with: Users, Doctor, ClinicSchedule, ClinicService, ClinicDocument, Appointment

```
CREATE TABLE dbo.[Clinic]
(
    [Id] INT IDENTITY(1,1) NOT NULL,
    [OwnerUserId] INT NOT NULL,
    [Name] NVARCHAR(150) NOT NULL,
    [ClinicType] NVARCHAR(50) NULL,
    [Phone] NVARCHAR(20) NOT NULL,
    [Email] NVARCHAR(150) NULL,
    [Description] NVARCHAR(1000) NULL,
    [City] NVARCHAR(100) NOT NULL,
    [Area] NVARCHAR(100) NULL,
    [Address] NVARCHAR(300) NULL,
    [Latitude] DECIMAL(9,6) NULL,
    [Longitude] DECIMAL(9,6) NULL,
    [DefaultOpenTime] TIME NULL,
    [DefaultCloseTime] TIME NULL,
    [HasEmergency] BIT NOT NULL DEFAULT (0),
    [VerificationStatus] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
    [Rating] DECIMAL(3,2) NOT NULL DEFAULT (0.00),
    [LogoUrl] NVARCHAR(500) NULL,
    [CoverImageUrl] NVARCHAR(500) NULL,
    [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT PK_Clinic PRIMARY KEY ([Id]),
    CONSTRAINT UQ_Clinic_OwnerUserId UNIQUE ([OwnerUserId]),
    CONSTRAINT FK_Clinic_OwnerUser FOREIGN KEY ([OwnerUserId]) REFERENCES dbo.[Users]([Id]),
    CONSTRAINT CK_Clinic_DefaultOpenClose CHECK (
        [DefaultOpenTime] IS NULL OR [DefaultCloseTime] IS NULL OR [DefaultOpenTime] <
[DefaultCloseTime]
    ),
    CONSTRAINT CK_Clinic_VerificationStatus CHECK ([VerificationStatus] IN (N'Pending',
N'Approved', N'Rejected', N'Suspended')),
    CONSTRAINT CK_Clinic_Rating CHECK ([Rating] >= 0 AND [Rating] <= 5)
);
```

ClinicSchedule:

Its function: Clinic opening hours for each day.

Associates with: Clinic

```
CREATE TABLE dbo.[ClinicSchedule]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
```

```
 [ClinicId] INT NOT NULL,
 [DayOfWeek] INT NOT NULL,
 [OpenTime] TIME NOT NULL,
 [CloseTime] TIME NOT NULL,
 [IsWorkingDay] BIT NOT NULL DEFAULT (1),
 CONSTRAINT PK_ClinicSchedule PRIMARY KEY ([Id]),
 CONSTRAINT UQ_ClinicSchedule_ClinicId_DayOfWeek UNIQUE ([ClinicId], [DayOfWeek]),
 CONSTRAINT FK_ClinicSchedule_Clinic FOREIGN KEY ([ClinicId]) REFERENCES dbo.[Clinic]([Id]),
 CONSTRAINT CK_ClinicSchedule_DayOfWeek CHECK ([DayOfWeek] BETWEEN 0 AND 6),
 CONSTRAINT CK_ClinicSchedule_OpenClose CHECK ([OpenTime] < [CloseTime])
);
```

ClinicService:

Its function: General services provided by the clinic.

Attached to: Clinic.

```
CREATE TABLE dbo.[ClinicSchedule]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [ClinicId] INT NOT NULL,
 [DayOfWeek] INT NOT NULL,
 [OpenTime] TIME NOT NULL,
 [CloseTime] TIME NOT NULL,
 [IsWorkingDay] BIT NOT NULL DEFAULT (1),
 CONSTRAINT PK_ClinicSchedule PRIMARY KEY ([Id]),
 CONSTRAINT UQ_ClinicSchedule_ClinicId_DayOfWeek UNIQUE ([ClinicId], [DayOfWeek]),
 CONSTRAINT FK_ClinicSchedule_Clinic FOREIGN KEY ([ClinicId]) REFERENCES dbo.[Clinic]([Id]),
 CONSTRAINT CK_ClinicSchedule_DayOfWeek CHECK ([DayOfWeek] BETWEEN 0 AND 6),
 CONSTRAINT CK_ClinicSchedule_OpenClose CHECK ([OpenTime] < [CloseTime])
);
```

ClinicDocument:

Its function: Clinic documentation files.

It is associated with: Clinic

```
CREATE TABLE dbo.[ClinicDocument]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [ClinicId] INT NOT NULL,
 [DocumentType] NVARCHAR(50) NOT NULL,
 [FileUrl] NVARCHAR(500) NOT NULL,
 [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
 [ExpiryDate] DATE NULL,
 [UploadedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_ClinicDocument PRIMARY KEY ([Id]),
```

```
 CONSTRAINT FK_ClinicDocument_Clinic FOREIGN KEY ([ClinicId]) REFERENCES dbo.[Clinic]([Id]),
 CONSTRAINT CK_ClinicDocument_Status CHECK ([Status] IN (N'Pending', N'Approved', N'Rejected',
N'Expired'))
);
```

Doctor:

Its function: Doctor data associated with a clinic.

It is linked to: Users, Clinic, DoctorService, DoctorDocument, Appointment.

Important note: According to the project's decision, doctors no longer log in with an invitation code; instead, the clinic creates it.

```
CREATE TABLE dbo.[Doctor]
(
    [Id] INT IDENTITY(1,1) NOT NULL,
    [UserId] INT NOT NULL,
    [ClinicId] INT NOT NULL,
    [Specialization] NVARCHAR(100) NOT NULL,
    [SecondarySpec] NVARCHAR(100) NULL,
    [ExperienceYears] INT NULL,
    [Qualification] NVARCHAR(300) NULL,
    [Certifications] NVARCHAR(1000) NULL,
    [Bio] NVARCHAR(500) NULL,
    [Languages] NVARCHAR(200) NULL,
    [WorkingDays] NVARCHAR(50) NULL,
    [StartTime] TIME NULL,
    [EndTime] TIME NULL,
    [AppointmentDuration] INT NOT NULL DEFAULT (30),
    [MaxDailyAppointments] INT NOT NULL DEFAULT (12),
    [IsAvailable] BIT NOT NULL DEFAULT (1),
    [VerificationStatus] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
    [Rating] DECIMAL(3,2) NOT NULL DEFAULT (0.00),
    [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT PK_Doctor PRIMARY KEY ([Id]),
    CONSTRAINT UQ_Doctor_UserId UNIQUE ([UserId]),
    CONSTRAINT FK_Doctor_User FOREIGN KEY ([UserId]) REFERENCES dbo.[Users]([Id]),
    CONSTRAINT FK_Doctor_Clinic FOREIGN KEY ([ClinicId]) REFERENCES dbo.[Clinic]([Id]),
    CONSTRAINT CK_Doctor_ExperienceYears CHECK ([ExperienceYears] IS NULL OR [ExperienceYears]
>= 0),
    CONSTRAINT CK_Doctor_AppointmentDuration CHECK ([AppointmentDuration] > 0),
    CONSTRAINT CK_Doctor_MaxDailyAppointments CHECK ([MaxDailyAppointments] > 0),
    CONSTRAINT CK_Doctor_Shift CHECK ([StartTime] IS NULL OR [EndTime] IS NULL OR [StartTime]
< [EndTime]),
    CONSTRAINT CK_Doctor_VerificationStatus CHECK ([VerificationStatus] IN (N'Pending',
N'Approved', N'Rejected')),
    CONSTRAINT CK_Doctor_Rating CHECK ([Rating] >= 0 AND [Rating] <= 5)
);
```

DoctorService:

Its function: Services and prices for each doctor.

It is associated with: Doctor.

```
CREATE TABLE dbo.[DoctorService]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [DoctorId] INT NOT NULL,
 [ServiceName] NVARCHAR(100) NOT NULL,
 [Price] DECIMAL(10,2) NULL,
 [IsActive] BIT NOT NULL DEFAULT (1),
 CONSTRAINT PK_DoctorService PRIMARY KEY ([Id]),
 CONSTRAINT FK_DoctorService_Doctor FOREIGN KEY ([DoctorId]) REFERENCES dbo.[Doctor]([Id]),
 CONSTRAINT CK_DoctorService_Price CHECK ([Price] IS NULL OR [Price] >= 0)
);
```

DoctorDocument:

Its function: Documenting and verifying the doctor's credentials.

Associated with: Doctor

```
CREATE TABLE dbo.[DoctorDocument]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [DoctorId] INT NOT NULL,
 [DocumentType] NVARCHAR(50) NOT NULL,
 [FileUrl] NVARCHAR(500) NOT NULL,
 [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
 [ExpiryDate] DATE NULL,
 [UploadedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_DoctorDocument PRIMARY KEY ([Id]),
 CONSTRAINT FK_DoctorDocument_Doctor FOREIGN KEY ([DoctorId]) REFERENCES dbo.[Doctor]([Id]),
 CONSTRAINT CK_DoctorDocument_Status CHECK ([Status] IN (N'Pending', N'Approved', N'Rejected',
N'Expired'))
);
```

Appointment:

Function: Medical bookings.

Associates with: Users, Pet, Doctor, Clinic.

Important Note: There is a default fixed BookingDeposit of 5.00 JOD, and its status is Held/Refunded/Forfeited depending on attendance.

```
CREATE TABLE dbo.[Appointment]
(
    [Id] INT IDENTITY(1,1) NOT NULL,
    [UserId] INT NOT NULL,
    [PetId] INT NOT NULL,
    [DoctorId] INT NOT NULL,
    [ClinicId] INT NOT NULL,
    [AppointmentDate] DATE NOT NULL,
    [StartTime] TIME NOT NULL,
    [EndTime] TIME NOT NULL,
    [ServiceName] NVARCHAR(100) NULL,
    [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
    [AppointmentType] NVARCHAR(30) NULL,
    [UrgencyLevel] NVARCHAR(20) NOT NULL DEFAULT (N'Normal'),
    [Reason] NVARCHAR(500) NULL,
    [Notes] NVARCHAR(500) NULL,
    [ConsultationFee] DECIMAL(10,2) NULL,
    [PaymentMethod] NVARCHAR(20) NULL,
    [PaymentStatus] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
    [BookingDeposit] DECIMAL(10,2) NOT NULL DEFAULT (5.00),
    [DepositStatus] NVARCHAR(20) NOT NULL DEFAULT (N'Held'),
    [CancellationReason] NVARCHAR(300) NULL,
    [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT PK_Appointment PRIMARY KEY ([Id]),
    CONSTRAINT FK_Appointment_User FOREIGN KEY ([UserId]) REFERENCES dbo.[Users]([Id]),
    CONSTRAINT FK_Appointment_Pet FOREIGN KEY ([PetId]) REFERENCES dbo.[Pet]([Id]),
    CONSTRAINT FK_Appointment_Doctor FOREIGN KEY ([DoctorId]) REFERENCES dbo.[Doctor]([Id]),
    CONSTRAINT FK_Appointment_Clinic FOREIGN KEY ([ClinicId]) REFERENCES dbo.[Clinic]([Id]),
    CONSTRAINT CK_Appointment_Time CHECK ([StartTime] < [EndTime]),
    CONSTRAINT CK_Appointment_Fees CHECK (([ConsultationFee] IS NULL OR [ConsultationFee] >=
0) AND [BookingDeposit] >= 0),
    CONSTRAINT CK_Appointment_Status CHECK ([Status] IN (N'Pending', N'Confirmed',
N'Rejected', N'Completed', N'Cancelled', N'NoShow')),
    CONSTRAINT CK_Appointment_PaymentStatus CHECK ([PaymentStatus] IN (N'Pending', N'Paid',
N'Refunded')),
    CONSTRAINT CK_Appointment_DepositStatus CHECK ([DepositStatus] IN (N'Held', N'Refunded',
N'Forfeited')),
    CONSTRAINT CK_Appointment_UrgencyLevel CHECK ([UrgencyLevel] IN (N'Normal', N'Urgent',
N'Emergency'))
);
```

Store:

Its function: Store data.

It is associated with: Users, StoreDocument, Category, Product, Order, Complaint.

```
CREATE TABLE dbo.[Store]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [OwnerUserId] INT NOT NULL,
 [Name] NVARCHAR(150) NOT NULL,
 [StoreType] NVARCHAR(100) NULL,
 [Description] NVARCHAR(1000) NULL,
 [LogoUrl] NVARCHAR(500) NULL,
 [CoverImageUrl] NVARCHAR(500) NULL,
 [Email] NVARCHAR(150) NULL,
 [Phone] NVARCHAR(20) NULL,
 [WhatsApp] NVARCHAR(20) NULL,
 [Country] NVARCHAR(100) NOT NULL DEFAULT (N'الأردن'),
 [City] NVARCHAR(100) NOT NULL,
 [Area] NVARCHAR(100) NULL,
 [Address] NVARCHAR(300) NULL,
 [Latitude] DECIMAL(9,6) NULL,
 [Longitude] DECIMAL(9,6) NULL,
 [DeliveryAvailable] BIT NOT NULL DEFAULT (0),
 [DeliveryAreas] NVARCHAR(500) NULL,
 [DeliveryFee] DECIMAL(10,2) NULL,
 [DeliveryTime] NVARCHAR(100) NULL,
 [OpenTime] TIME NULL,
 [CloseTime] TIME NULL,
 [WorkingDays] NVARCHAR(50) NULL,
 [AcceptedPayments] NVARCHAR(200) NULL,
 [VerificationStatus] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
 [VerifiedAt] DATETIME2 NULL,
 [VerifiedByAdminId] INT NULL,
 [Rating] DECIMAL(3,2) NOT NULL DEFAULT (0.00),
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_Store PRIMARY KEY ([Id]),
 CONSTRAINT UQ_Store_OwnerUserId UNIQUE ([OwnerUserId]),
 CONSTRAINT FK_Store_OwnerUser FOREIGN KEY ([OwnerUserId]) REFERENCES dbo.[Users]([Id]),
 CONSTRAINT FK_Store_VerifiedByAdmin FOREIGN KEY ([VerifiedByAdminId]) REFERENCES
dbo.[Users]([Id]),
 CONSTRAINT CK_Store_OpenClose CHECK ([OpenTime] IS NULL OR [CloseTime] IS NULL OR [OpenTime]
< [CloseTime]),
 CONSTRAINT CK_Store_DeliveryFee CHECK ([DeliveryFee] IS NULL OR [DeliveryFee] >= 0),
 CONSTRAINT CK_Store_VerificationStatus CHECK ([VerificationStatus] IN (N'Pending',
N'Approved', N'Rejected', N'Suspended')),
 CONSTRAINT CK_Store_Rating CHECK ([Rating] >= 0 AND [Rating] <= 5)
);
```

StoreDocument:

Its function: Store documentation.

Associates with: Store.

```
CREATE TABLE dbo.[StoreDocument]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [StoreId] INT NOT NULL,
 [DocumentType] NVARCHAR(50) NOT NULL,
 [FileUrl] NVARCHAR(500) NOT NULL,
 [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
 [ExpiryDate] DATE NULL,
 [UploadedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_StoreDocument PRIMARY KEY ([Id]),
 CONSTRAINT FK_StoreDocument_Store FOREIGN KEY ([StoreId]) REFERENCES dbo.[Store]([Id]),
 CONSTRAINT CK_StoreDocument_Status CHECK ([Status] IN (N'Pending', N'Approved', N'Rejected',
N'Expired'))
);
```

Category:

Its function: Product categorization for each store.

Associates with: Store, Product

```
CREATE TABLE dbo.[Category]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [StoreId] INT NOT NULL,
 [Name] NVARCHAR(100) NOT NULL,
 [Icon] NVARCHAR(10) NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_Category PRIMARY KEY ([Id]),
 CONSTRAINT UQ_Category_StoreId_Name UNIQUE ([StoreId], [Name]),
 CONSTRAINT FK_Category_Store FOREIGN KEY ([StoreId]) REFERENCES dbo.[Store]([Id])
);
```

Product:

Its function: Store products.

Associated with: Store, Category, OrderItem, CartItem

```
CREATE TABLE dbo.[Product]
(
    [Id] INT IDENTITY(1,1) NOT NULL,
    [StoreId] INT NOT NULL,
    [CategoryId] INT NULL,
    [Name] NVARCHAR(200) NOT NULL,
    [Description] NVARCHAR(2000) NULL,
```

```
    [Price] DECIMAL(10,2) NOT NULL,
    [SalePrice] DECIMAL(10,2) NULL,
    [StockQuantity] INT NOT NULL DEFAULT (0),
    [PetType] NVARCHAR(100) NULL,
    [ImageUrl] NVARCHAR(500) NULL,
    [IsVisible] BIT NOT NULL DEFAULT (1),
    [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT PK_Product PRIMARY KEY ([Id]),
    CONSTRAINT FK_Product_Store FOREIGN KEY ([StoreId]) REFERENCES dbo.[Store]([Id]),
    CONSTRAINT FK_Product_Category FOREIGN KEY ([CategoryId]) REFERENCES dbo.[Category]([Id]),
    CONSTRAINT CK_Product_Price CHECK ([Price] >= 0),
    CONSTRAINT CK_Product_SalePrice CHECK ([SalePrice] IS NULL OR ([SalePrice] >= 0 AND
[SalePrice] <= [Price])),
    CONSTRAINT CK_Product_StockQuantity CHECK ([StockQuantity] >= 0)
);
```

Order:

Its function: Order header.

It is associated with: Users, Store, OrderItem.

```
CREATE TABLE dbo.[Order]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [UserId] INT NOT NULL,
 [StoreId] INT NOT NULL,
 [OrderCode] NVARCHAR(30) NULL,
 [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
 [PaymentMethod] NVARCHAR(20) NULL,
 [PaymentStatus] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
 [RecipientName] NVARCHAR(100) NULL,
 [RecipientPhone] NVARCHAR(20) NULL,
 [DeliveryAddress] NVARCHAR(300) NULL,
 [DeliveryFee] DECIMAL(10,2) NULL,
 [Notes] NVARCHAR(500) NULL,
 [CancellationReason] NVARCHAR(300) NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_Order PRIMARY KEY ([Id]),
 CONSTRAINT UQ_Order_OrderCode UNIQUE ([OrderCode]),
 CONSTRAINT FK_Order_User FOREIGN KEY ([UserId]) REFERENCES dbo.[Users]([Id]),
 CONSTRAINT FK_Order_Store FOREIGN KEY ([StoreId]) REFERENCES dbo.[Store]([Id]),
 CONSTRAINT CK_Order_DeliveryFee CHECK ([DeliveryFee] IS NULL OR [DeliveryFee] >= 0),
 CONSTRAINT CK_Order_Status CHECK ([Status] IN (N'Pending', N'Confirmed', N'Preparing',
N'Shipped', N'Delivered', N'Cancelled')),
 CONSTRAINT CK_Order_PaymentStatus CHECK ([PaymentStatus] IN (N'Pending', N'Paid', N'Failed',
N'Refunded'))
);
```

OrderItem:

Its function: Order items.

Associated with: Order, Product.

Note: The ProductNameSnapshot is important so that the product name remains saved even if it changes later.

```
CREATE TABLE dbo.[OrderItem]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [OrderId] INT NOT NULL,
 [ProductId] INT NOT NULL,
 [ProductNameSnapshot] NVARCHAR(200) NOT NULL,
 [Quantity] INT NOT NULL,
 [UnitPrice] DECIMAL(10,2) NOT NULL,
 [SalePrice] DECIMAL(10,2) NULL,
 CONSTRAINT PK_OrderItem PRIMARY KEY ([Id]),
 CONSTRAINT FK_OrderItem_Order FOREIGN KEY ([OrderId]) REFERENCES dbo.[Order]([Id]),
 CONSTRAINT FK_OrderItem_Product FOREIGN KEY ([ProductId]) REFERENCES dbo.[Product]([Id]),
 CONSTRAINT CK_OrderItem_Quantity CHECK ([Quantity] > 0),
 CONSTRAINT CK_OrderItem_UnitPrice CHECK ([UnitPrice] >= 0),
 CONSTRAINT CK_OrderItem_SalePrice CHECK ([SalePrice] IS NULL OR ([SalePrice] >= 0 AND [SalePrice] <= [UnitPrice]))
);
```

Cart:

Its function: User basket.

It is associated with: Users, CartItem.

```
CREATE TABLE dbo.[Cart]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [UserId] INT NOT NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_Cart PRIMARY KEY ([Id]),
 CONSTRAINT UQ_Cart_UserId UNIQUE ([UserId]),
 CONSTRAINT FK_Cart_User FOREIGN KEY ([UserId]) REFERENCES dbo.[Users]([Id])
);
```

CartItem:

Its function: Items inside the basket.

Associates with: Cart, Product

```
CREATE TABLE dbo.[CartItem]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [CartId] INT NOT NULL,
 [ProductId] INT NOT NULL,
 [Quantity] INT NOT NULL,
 CONSTRAINT PK_CartItem PRIMARY KEY ([Id]),
 CONSTRAINT UQ_CartItem_CartId_ProductId UNIQUE ([CartId], [ProductId]),
 CONSTRAINT FK_CartItem_Cart FOREIGN KEY ([CartId]) REFERENCES dbo.[Cart]([Id]),
 CONSTRAINT FK_CartItem_Product FOREIGN KEY ([ProductId]) REFERENCES dbo.[Product]([Id]),
 CONSTRAINT CK_CartItem_Quantity CHECK ([Quantity] > 0)
);
```

SubscriptionPlan:

Its function: Subscription plans.

Associated with: Subscription.

```
CREATE TABLE dbo.[SubscriptionPlan]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [Name] NVARCHAR(50) NOT NULL,
 [Price] DECIMAL(10,2) NOT NULL,
 [DurationInDays] INT NOT NULL,
 [MaxProducts] INT NULL,
 [MaxDoctors] INT NULL,
 [MaxAppointments] INT NULL,
 [CommissionRate] DECIMAL(5,2) NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_SubscriptionPlan PRIMARY KEY ([Id]),
 CONSTRAINT CK_SubscriptionPlan_Price CHECK ([Price] >= 0),
 CONSTRAINT CK_SubscriptionPlan_DurationInDays CHECK ([DurationInDays] > 0),
 CONSTRAINT CK_SubscriptionPlan_MaxProducts CHECK ([MaxProducts] IS NULL OR [MaxProducts] >=
0),
 CONSTRAINT CK_SubscriptionPlan_MaxDoctors CHECK ([MaxDoctors] IS NULL OR [MaxDoctors] >= 0),
 CONSTRAINT CK_SubscriptionPlan_MaxAppointments CHECK ([MaxAppointments] IS NULL OR
[MaxAppointments] >= 0),
 CONSTRAINT CK_SubscriptionPlan_CommissionRate CHECK ([CommissionRate] IS NULL OR
([CommissionRate] BETWEEN 0 AND 100))
);
```

Subscription:

Its function: Registers subscriptions for a store or clinic.

It is linked to: SubscriptionPlan.

Important note: EntityType + EntityId are polymorphic relationships, therefore there is no direct FK on Clinic/Store. Verification is done from the backend.

```
CREATE TABLE dbo.[Subscription]
(
    [Id] INT IDENTITY(1,1) NOT NULL,
    [PlanId] INT NOT NULL,
    [EntityType] NVARCHAR(20) NOT NULL,
    [EntityId] INT NOT NULL,
    [StartDate] DATETIME2 NOT NULL,
    [EndDate] DATETIME2 NOT NULL,
    [AmountPaid] DECIMAL(10,2) NULL,
    [PaymentStatus] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
    [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Active'),
    [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT PK_Subscription PRIMARY KEY ([Id]),
    CONSTRAINT FK_Subscription_Plan FOREIGN KEY ([PlanId]) REFERENCES
dbo.[SubscriptionPlan]([Id]),
    CONSTRAINT CK_Subscription_EntityType CHECK ([EntityType] IN (N'Clinic', N'Store')),
    CONSTRAINT CK_Subscription_DateRange CHECK ([StartDate] < [EndDate]),
    CONSTRAINT CK_Subscription_AmountPaid CHECK ([AmountPaid] IS NULL OR [AmountPaid] >= 0),
    CONSTRAINT CK_Subscription_PaymentStatus CHECK ([PaymentStatus] IN (N'Pending', N'Paid',
N'Failed', N'Refunded')),
    CONSTRAINT CK_Subscription_Status CHECK ([Status] IN (N'Active', N'Expired',
N'Cancelled'))
);
```

Complaint:

Its function: Primary complaint.

It is associated with: Users, ComplaintReceiver, ComplaintReply.

Note: If TargetType = System, the TargetId must be NULL; if Clinic or Store, a TargetId is required.

```
CREATE TABLE dbo.[Complaint]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [UserId] INT NOT NULL,
 [TargetType] NVARCHAR(20) NOT NULL,
 [TargetId] INT NULL,
 [Title] NVARCHAR(200) NOT NULL,
 [Description] NVARCHAR(1000) NOT NULL,
 [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Open'),
```

```
 [Priority] NVARCHAR(20) NOT NULL DEFAULT (N'Medium'),
 [ResolvedAt] DATETIME2 NULL,
 [ResolvedByAdminId] INT NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_Complaint PRIMARY KEY ([Id]),
 CONSTRAINT FK_Complaint_User FOREIGN KEY ([UserId]) REFERENCES dbo.[Users]([Id]),
 CONSTRAINT FK_Complaint_ResolvedByAdmin FOREIGN KEY ([ResolvedByAdminId]) REFERENCES
dbo.[Users]([Id]),
 CONSTRAINT CK_Complaint_TargetType CHECK ([TargetType] IN (N'Clinic', N'Store', N'System')),
 CONSTRAINT CK_Complaint_TargetIdRule CHECK (
 ([TargetType] = N'System' AND [TargetId] IS NULL)
 OR ([TargetType] IN (N'Clinic', N'Store') AND [TargetId] IS NOT NULL)
 ),
 CONSTRAINT CK_Complaint_Status CHECK ([Status] IN (N'Open', N'InProgress', N'Resolved',
N'Rejected')),
 CONSTRAINT CK_Complaint_Priority CHECK ([Priority] IN (N'Low', N'Medium', N'High'))
);
```

ComplaintReceiver:

Its function: It determines who a complaint should go to.

It is associated with: Complaint, Users.

Important note: This table is necessary because a complaint may go to more than one administrator, such as the department administrator plus the SuperAdmin.

```
CREATE TABLE dbo.[ComplaintReceiver]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [ComplaintId] INT NOT NULL,
 [ReceiverUserId] INT NOT NULL,
 [ReceiverRole] NVARCHAR(30) NOT NULL,
 [IsRead] BIT NOT NULL DEFAULT (0),
 [ReadAt] DATETIME2 NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_ComplaintReceiver PRIMARY KEY ([Id]),
 CONSTRAINT FK_ComplaintReceiver_Complaint FOREIGN KEY ([ComplaintId]) REFERENCES
dbo.[Complaint]([Id]),
 CONSTRAINT FK_ComplaintReceiver_User FOREIGN KEY ([ReceiverUserId]) REFERENCES
dbo.[Users]([Id]),
 CONSTRAINT CK_ComplaintReceiver_Role CHECK ([ReceiverRole] IN (N'ClinicAdmin', N'StoreAdmin',
N'SuperAdmin'))
);
```

ComplaintReply:

Its function: Admin responses to complaints.

It is associated with: Complaints, Users.

```
CREATE TABLE dbo.[ComplaintReply]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [ComplaintId] INT NOT NULL,
 [AdminId] INT NOT NULL,
 [Message] NVARCHAR(1000) NOT NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_ComplaintReply PRIMARY KEY ([Id]),
 CONSTRAINT FK_ComplaintReply_Complaint FOREIGN KEY ([ComplaintId]) REFERENCES
dbo.[Complaint]([Id]),
 CONSTRAINT FK_ComplaintReply_Admin FOREIGN KEY ([AdminId]) REFERENCES dbo.[Users]([Id])
);
```

LostPetReport:

Its function: Reporting a lost pet.

It connects to: Users, Pet, LostPetImage, FoundPetClaim.

Note: It includes snapshot data even if the pet is not linked to a pet record.

```
CREATE TABLE dbo.[LostPetReport]
(
    [Id] INT IDENTITY(1,1) NOT NULL,
    [UserId] INT NOT NULL,
    [PetId] INT NULL,
    [PetName] NVARCHAR(100) NULL,
    [AnimalType] NVARCHAR(50) NULL,
    [Breed] NVARCHAR(100) NULL,
    [Color] NVARCHAR(100) NULL,
    [LastSeenLocation] NVARCHAR(300) NOT NULL,
    [City] NVARCHAR(100) NOT NULL,
    [Area] NVARCHAR(100) NULL,
    [Latitude] DECIMAL(9,6) NULL,
    [Longitude] DECIMAL(9,6) NULL,
    [DateLost] DATE NOT NULL,
    [TimeLost] TIME NULL,
    [Description] NVARCHAR(1000) NULL,
    [SpecialMarks] NVARCHAR(500) NULL,
    [MicrochipId] NVARCHAR(50) NULL,
    [SecretQuestion] NVARCHAR(300) NOT NULL,
    [SecretAnswerHash] NVARCHAR(500) NOT NULL,
    [RewardAmount] DECIMAL(10,2) NULL,
    [RewardStatus] NVARCHAR(20) NULL,
    [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Active'),
    [ExpiryDate] DATETIME2 NULL,
    [ReportCode] NVARCHAR(20) NULL,
```

```
    [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT PK_LostPetReport PRIMARY KEY ([Id]),
    CONSTRAINT FK_LostPetReport_User FOREIGN KEY ([UserId]) REFERENCES dbo.[Users]([Id]),
    CONSTRAINT FK_LostPetReport_Pet FOREIGN KEY ([PetId]) REFERENCES dbo.[Pet]([Id]),
    CONSTRAINT CK_LostPetReport_RewardAmount CHECK ([RewardAmount] IS NULL OR [RewardAmount]
>= 0),
    CONSTRAINT CK_LostPetReport_Status CHECK ([Status] IN (N'Active', N'Claimed', N'Found',
N'Expired')),
    CONSTRAINT CK_LostPetReport_RewardStatus CHECK ([RewardStatus] IS NULL OR [RewardStatus]
IN (N'Frozen', N'Released'))
);
```

LostPetImage:

Its function: To report lost pets.

It is associated with: LostPetReport

```
CREATE TABLE dbo.[LostPetImage]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [ReportId] INT NOT NULL,
 [ImageUrl] NVARCHAR(500) NOT NULL,
 CONSTRAINT PK_LostPetImage PRIMARY KEY ([Id]),
 CONSTRAINT FK_LostPetImage_Report FOREIGN KEY ([ReportId]) REFERENCES
dbo.[LostPetReport]([Id])
);
```

FoundPetClaim:

Its function: Claiming to find the animal.

It is associated with: LostPetReport, Users, FoundPetImage, ClaimAttemptLog.

```
CREATE TABLE dbo.[FoundPetClaim]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [LostReportId] INT NOT NULL,
 [FinderId] INT NOT NULL,
 [FoundLocation] NVARCHAR(300) NULL,
 [City] NVARCHAR(100) NULL,
 [Area] NVARCHAR(100) NULL,
 [Latitude] DECIMAL(9,6) NULL,
 [Longitude] DECIMAL(9,6) NULL,
 [FoundDate] DATE NOT NULL,
 [FoundTime] TIME NULL,
 [Description] NVARCHAR(1000) NULL,
 [AnimalType] NVARCHAR(50) NULL,
```

```
 [Color] NVARCHAR(100) NULL,
 [SpecialMarks] NVARCHAR(500) NULL,
 [MatchScore] DECIMAL(5,2) NULL,
 [AttemptsCount] INT NOT NULL DEFAULT (0),
 [FinderCode] NVARCHAR(20) NULL,
 [OwnerCode] NVARCHAR(20) NULL,
 [CodesExchanged] BIT NOT NULL DEFAULT (0),
 [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
 [ExpiresAt] DATETIME2 NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_FoundPetClaim PRIMARY KEY ([Id]),
 CONSTRAINT FK_FoundPetClaim_LostReport FOREIGN KEY ([LostReportId]) REFERENCES
dbo.[LostPetReport]([Id]),
 CONSTRAINT FK_FoundPetClaim_Finder FOREIGN KEY ([FinderId]) REFERENCES dbo.[Users]([Id]),
 CONSTRAINT CK_FoundPetClaim_MatchScore CHECK ([MatchScore] IS NULL OR ([MatchScore] >= 0 AND
[MatchScore] <= 100)),
 CONSTRAINT CK_FoundPetClaim_AttemptsCount CHECK ([AttemptsCount] >= 0),
 CONSTRAINT CK_FoundPetClaim_Status CHECK ([Status] IN (N'Pending', N'Verified', N'Rejected',
N'Locked'))
);
```

FoundPetImage:

Its function: To claim photos of the animal being found.

Associated with: FoundPetClaim

```
CREATE TABLE dbo.[FoundPetImage]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [ClaimId] INT NOT NULL,
 [ImageUrl] NVARCHAR(500) NOT NULL,
 CONSTRAINT PK_FoundPetImage PRIMARY KEY ([Id]),
 CONSTRAINT FK_FoundPetImage_Claim FOREIGN KEY ([ClaimId]) REFERENCES
dbo.[FoundPetClaim]([Id])
);
```

ClaimAttemptLog:

Its function: To record attempts to verify the secret answer.

It is associated with: FoundPetClaim.

```
CREATE TABLE dbo.[ClaimAttemptLog]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [ClaimId] INT NOT NULL,
 [AttemptValue] NVARCHAR(300) NOT NULL,
 [IsCorrect] BIT NOT NULL,
 [IPAddress] NVARCHAR(50) NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
```

```
 CONSTRAINT PK_ClaimAttemptLog PRIMARY KEY ([Id]),
 CONSTRAINT FK_ClaimAttemptLog_Claim FOREIGN KEY ([ClaimId]) REFERENCES
dbo.[FoundPetClaim]([Id])
);
```

DangerousAnimalReport:

Its function: To report a dangerous or injured animal or similar problem.

Associates with: Users, DangerousAnimalImage.

```
CREATE TABLE dbo.[DangerousAnimalReport]
(
    [Id] INT IDENTITY(1,1) NOT NULL,
    [ReporterId] INT NOT NULL,
    [AnimalType] NVARCHAR(50) NOT NULL,
    [ProblemType] NVARCHAR(100) NOT NULL,
    [Severity] NVARCHAR(20) NOT NULL,
    [Description] NVARCHAR(1000) NOT NULL,
    [City] NVARCHAR(100) NOT NULL,
    [Area] NVARCHAR(100) NULL,
    [LocationText] NVARCHAR(300) NULL,
    [Latitude] DECIMAL(9,6) NULL,
    [Longitude] DECIMAL(9,6) NULL,
    [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Open'),
    [ReportCode] NVARCHAR(20) NULL,
    [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT PK_DangerousAnimalReport PRIMARY KEY ([Id]),
    CONSTRAINT FK_DangerousAnimalReport_User FOREIGN KEY ([ReporterId]) REFERENCES
dbo.[Users]([Id]),
    CONSTRAINT CK_DangerousAnimalReport_Severity CHECK ([Severity] IN (N'Low', N'Medium',
N'High')),
    CONSTRAINT CK_DangerousAnimalReport_Status CHECK ([Status] IN (N'Open', N'Processing',
N'Resolved', N'Fake'))
);
```

DangerousAnimalImage:

Its function: To report dangerous animals.

Related to: DangerousAnimalReport

```
CREATE TABLE dbo.[DangerousAnimalImage]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [ReportId] INT NOT NULL,
 [ImageUrl] NVARCHAR(500) NOT NULL,
 CONSTRAINT PK_DangerousAnimalImage PRIMARY KEY ([Id]),
 CONSTRAINT FK_DangerousAnimalImage_Report FOREIGN KEY ([ReportId]) REFERENCES
dbo.[DangerousAnimalReport]([Id])
);
```

AdoptionPost:

Its function: Adoption post.

It is associated with: Pet, AdoptionImage.

Important note: PostedByType + PostedById are polymorphic relationships, so verification is done from the backend, not directly from the FK.

```
CREATE TABLE dbo.[AdoptionPost]
(
    [Id] INT IDENTITY(1,1) NOT NULL,
    [PostedByType] NVARCHAR(20) NOT NULL,
    [PostedById] INT NOT NULL,
    [PetId] INT NULL,
    [PetName] NVARCHAR(100) NULL,
    [AnimalType] NVARCHAR(50) NULL,
    [Breed] NVARCHAR(100) NULL,
    [Color] NVARCHAR(100) NULL,
    [Age] INT NULL,
    [Gender] NVARCHAR(10) NULL,
    [EnergyLevel] NVARCHAR(20) NULL,
    [GoodWithChildren] BIT NULL,
    [GoodWithPets] BIT NULL,
    [HealthStatus] NVARCHAR(200) NULL,
    [IsVaccinated] BIT NULL,
    [IsNeutered] BIT NULL,
    [City] NVARCHAR(100) NULL,
    [Area] NVARCHAR(100) NULL,
    [ContactName] NVARCHAR(100) NULL,
    [ContactPhone] NVARCHAR(20) NULL,
    [Description] NVARCHAR(1000) NULL,
    [ReasonForAdoption] NVARCHAR(500) NULL,
    [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
    [AdoptionStatus] NVARCHAR(20) NOT NULL DEFAULT (N'Available'),
    [PostCode] NVARCHAR(20) NULL,
    [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT PK_AdoptionPost PRIMARY KEY ([Id]),
    CONSTRAINT FK_AdoptionPost_Pet FOREIGN KEY ([PetId]) REFERENCES dbo.[Pet]([Id]),
    CONSTRAINT CK_AdoptionPost_PostedByType CHECK ([PostedByType] IN (N'User', N'Clinic',
N'Store')),
    CONSTRAINT CK_AdoptionPost_Age CHECK ([Age] IS NULL OR [Age] >= 0),
    CONSTRAINT CK_AdoptionPost_Status CHECK ([Status] IN (N'Pending', N'Approved',
N'Rejected')),
    CONSTRAINT CK_AdoptionPost_AdoptionStatus CHECK ([AdoptionStatus] IN (N'Available',
N'Reserved', N'Adopted'))
);
```

AdoptionImage:

Its function: Images of adoption posts.

It is associated with: AdoptionPost

```
CREATE TABLE dbo.[AdoptionImage]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [PostId] INT NOT NULL,
 [ImageUrl] NVARCHAR(500) NOT NULL,
 CONSTRAINT PK_AdoptionImage PRIMARY KEY ([Id]),
 CONSTRAINT FK_AdoptionImage_Post FOREIGN KEY ([PostId]) REFERENCES dbo.[AdoptionPost]([Id])
);
```

Donation:

Its function: Financial or in-kind donations.

It is associated with: Users.

Important note: TargetType + TargetId are polymorphic relationships; this should be verified in the backend.

```
CREATE TABLE dbo.[Donation]
(
    [Id] INT IDENTITY(1,1) NOT NULL,
    [DonorId] INT NOT NULL,
    [DonationType] NVARCHAR(20) NOT NULL,
    [Amount] DECIMAL(10,2) NULL,
    [Quantity] INT NULL,
    [Unit] NVARCHAR(50) NULL,
    [Description] NVARCHAR(500) NULL,
    [ProofImageUrl] NVARCHAR(500) NULL,
    [TargetType] NVARCHAR(20) NULL,
    [TargetId] INT NULL,
    [ReferenceCode] NVARCHAR(50) NULL,
    [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Pending'),
    [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT PK_Donation PRIMARY KEY ([Id]),
    CONSTRAINT FK_Donation_Donor FOREIGN KEY ([DonorId]) REFERENCES dbo.[Users]([Id]),
    CONSTRAINT CK_Donation_Amount CHECK ([Amount] IS NULL OR [Amount] >= 0),
    CONSTRAINT CK_Donation_Quantity CHECK ([Quantity] IS NULL OR [Quantity] >= 0),
    CONSTRAINT CK_Donation_DonationType CHECK ([DonationType] IN (N'Money', N'Food',
N'Medicine', N'Supplies')),
    CONSTRAINT CK_Donation_Status CHECK ([Status] IN (N'Pending', N'Approved',
```

```
N'Distributed'))
);
```

PlatformLaw:

Its function: Platform rules and policies.

It interacts with: Users.

```
CREATE TABLE dbo.[PlatformLaw]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [Title] NVARCHAR(200) NOT NULL,
 [Category] NVARCHAR(50) NOT NULL,
 [Content] NVARCHAR(MAX) NOT NULL,
 [Source] NVARCHAR(300) NULL,
 [IssuedDate] DATE NULL,
 [LastUpdated] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 [CreatedBy] INT NOT NULL,
 CONSTRAINT PK_PlatformLaw PRIMARY KEY ([Id]),
 CONSTRAINT FK_PlatformLaw_CreatedBy FOREIGN KEY ([CreatedBy]) REFERENCES dbo.[Users]([Id])
);
```

Notification:

Its function: Notifications.

It is associated with: Users.

Important note: RelatedEntityType + RelatedEntityId are polymorphic relationships, so verifying them requires backend data.

```
CREATE TABLE dbo.[Notification]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [UserId] INT NOT NULL,
 [Type] NVARCHAR(50) NOT NULL,
 [Title] NVARCHAR(200) NOT NULL,
 [Message] NVARCHAR(500) NOT NULL,
 [IsRead] BIT NOT NULL DEFAULT (0),
 [ReadAt] DATETIME2 NULL,
 [RelatedEntityType] NVARCHAR(50) NULL,
 [RelatedEntityId] INT NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_Notification PRIMARY KEY ([Id]),
 CONSTRAINT FK_Notification_User FOREIGN KEY ([UserId]) REFERENCES dbo.[Users]([Id])
);
```

Review:

Its function: Reviews for clinics, doctors, or stores.

It's associated with: Users.

Important note: TargetType + TargetId are polymorphic relationships, so there is no direct FK on Clinic/Doctor/Store. Verification is done in the backend.

```
CREATE TABLE dbo.[Review]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [UserId] INT NOT NULL,
 [TargetType] NVARCHAR(20) NOT NULL,
 [TargetId] INT NOT NULL,
 [Rating] INT NOT NULL,
 [Comment] NVARCHAR(1000) NULL,
 [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Visible'),
 [UpdatedAt] DATETIME2 NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_Review PRIMARY KEY ([Id]),
 CONSTRAINT UQ_Review_User_Target UNIQUE ([UserId], [TargetType], [TargetId]),
 CONSTRAINT FK_Review_User FOREIGN KEY ([UserId]) REFERENCES dbo.[Users]([Id]),
 CONSTRAINT CK_Review_TargetType CHECK ([TargetType] IN (N'Clinic', N'Doctor', N'Store')),
 CONSTRAINT CK_Review_Rating CHECK ([Rating] BETWEEN 1 AND 5),
 CONSTRAINT CK_Review_Status CHECK ([Status] IN (N'Visible', N'Hidden'))
);
```

ReportActionLog:

Its function: An administrative record of actions taken on complaints and reports.

It is associated with: Users.

Important note: EntityType + EntityId are polymorphic relationships; verify this in the backend.

```
CREATE TABLE dbo.[ReportActionLog]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [EntityType] NVARCHAR(50) NOT NULL,
 [EntityId] INT NOT NULL,
 [ActionType] NVARCHAR(50) NOT NULL,
 [StatusAfterAction] NVARCHAR(20) NULL,
 [Note] NVARCHAR(500) NULL,
 [AdminId] INT NOT NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_ReportActionLog PRIMARY KEY ([Id]),
 CONSTRAINT FK_ReportActionLog_Admin FOREIGN KEY ([AdminId]) REFERENCES dbo.[Users]([Id])
);
```

AdminMapMarker:

Its function: Management map markers.

It associates with: Users.

Important note: ReferenceType + ReferenceId are polymorphic relationships; there is no direct FK.

```
CREATE TABLE dbo.[AdminMapMarker]
(
 [Id] INT IDENTITY(1,1) NOT NULL,
 [MarkerType] NVARCHAR(50) NOT NULL,
 [ReferenceType] NVARCHAR(50) NULL,
 [ReferenceId] INT NULL,
 [Latitude] DECIMAL(9,6) NOT NULL,
 [Longitude] DECIMAL(9,6) NOT NULL,
 [Status] NVARCHAR(20) NOT NULL DEFAULT (N'Active'),
 [CreatedByAdminId] INT NOT NULL,
 [CreatedAt] DATETIME2 NOT NULL DEFAULT (GETDATE()),
 CONSTRAINT PK_AdminMapMarker PRIMARY KEY ([Id]),
 CONSTRAINT FK_AdminMapMarker_Admin FOREIGN KEY ([CreatedByAdminId]) REFERENCES
dbo.[Users]([Id]),
 CONSTRAINT CK_AdminMapMarker_Status CHECK ([Status] IN (N'Active', N'Hidden', N'Resolved'))
);
```

---

1) Relationships that need to be examined back-end

- Subscription(EntityType, EntityId)
- WalletTransaction(ReferenceType, ReferenceId)
- Review(TargetType, TargetId)
- AdoptionPost(PostedByType, PostedById)
- Donation(TargetType, TargetId)
- Notification(RelatedEntityType, RelatedEntityId)
- AdminMapMarker(ReferenceType, ReferenceId)
- ReportActionLog(EntityType, EntityId)